Hibernate Introduction

Hibernate is a powerful Java framework that simplifies database interactions by providing an Object-Relational Mapping (ORM) solution. It maps Java objects to database tables, allowing developers to work with data in an object-oriented manner without writing complex SQL queries.

Prerequisites for Learning Hibernate

1. ***Java**:*
    - Strong understanding of Java fundamentals, including OOP concepts.
    - Familiarity with Java Collections, Exceptions, and Interfaces.
2. ***DBMS (Database Management Systems)**:*
    - Basic knowledge of relational databases (e.g., MySQL, PostgreSQL).
    - Understanding how data is stored, retrieved, and manipulated.
3. ***SQL (Structured Query Language)**:*
    - Ability to write and understand basic to intermediate SQL queries.
4. ***JDBC (Java Database Connectivity)**:*
    - Knowledge of setting up database connections in Java.
    - Understanding how to execute SQL queries and process results using JDBC.

Fundamentals Before Hibernate

Persistence:

- ***Definition****:* The process of storing and managing data for a long time is called persistence.
- ***Examples****:*
    1. **File Operations**: Storing data in files on a hard disk (e.g., using Java I/O operations).
    2. **Database Storage**: Storing data in the form of tables in a database (e.g., using SQL).

Persistence Store:

- ***Definition****:* A persistence store is a place or storage where data is saved and managed for an extended period.

Persistent Data:

- ***Definition***: The data stored in a persistence store is referred to as persistence data.

Persistence Operations:

- ***Definition****:* The operations performed on persistent data, such as creating, reading, updating, or deleting data.

Persistence Logic:

- ***Definition****:* The logic written to perform persistence operations is called persistence logic. This logic ensures data is correctly stored and retrieved.

Persistence Technology/Framework:

- ***Definition****:* The technology or framework used to write persistence logic is known as persistence technology or framework.
- ***examples****:*
    - ***JDBC (Java Database Connectivity)****:* A technology for connecting and executing queries against the database.

- **Hibernate**: An ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java objects to database tables.
- **Spring JDBC / Spring ORM / Spring Data JPA**: Frameworks provided by Spring to manage persistence logic with advanced features and integration.

Additional Notes:

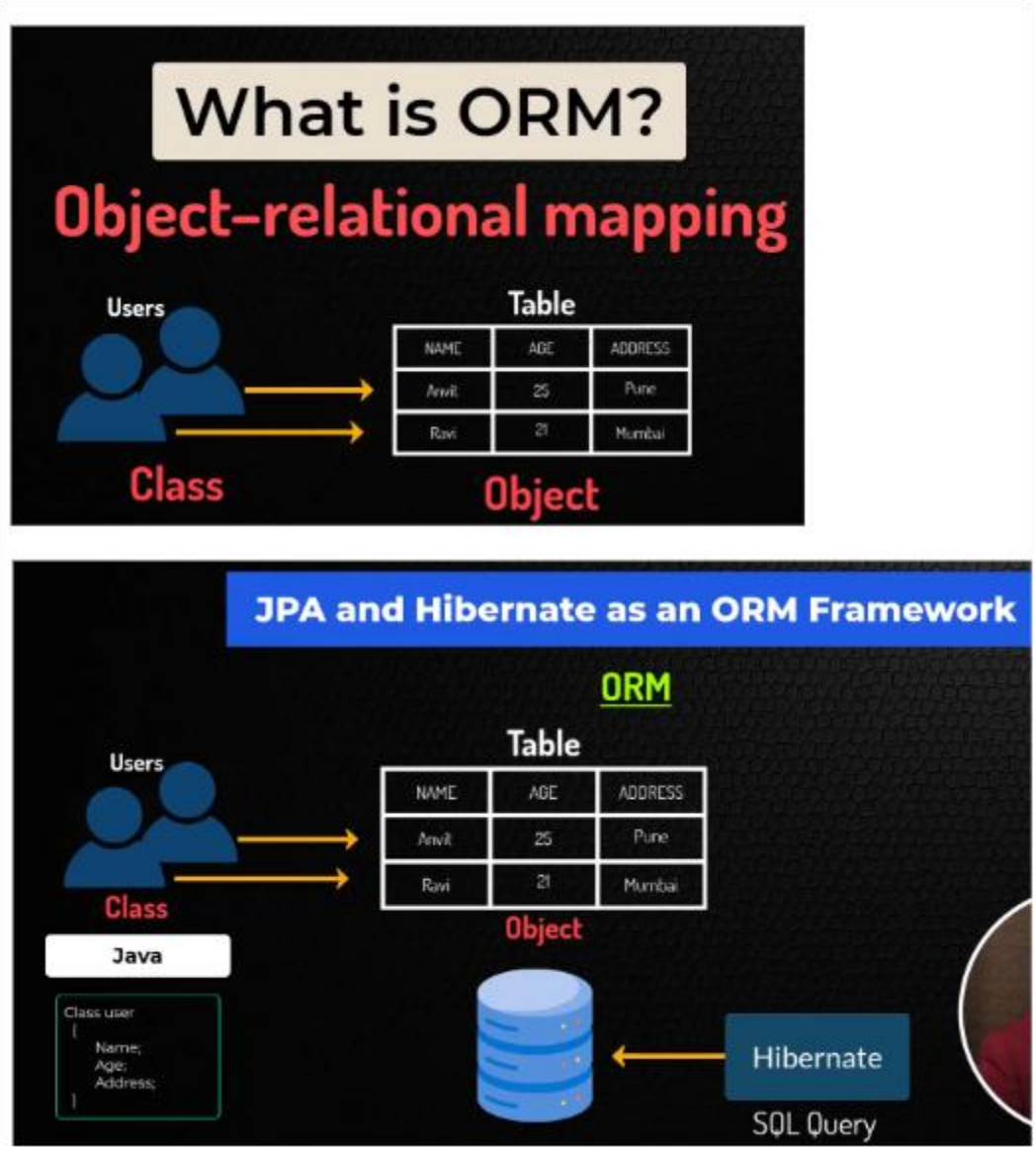
- ***Persistence in Modern Applications****:* Modern applications often require persistent data storage to ensure that data is not lost when the application is closed or the system is restarted. Technologies like Hibernate abstract the complexity of database interactions, allowing developers to focus on business logic.
- ***Importance of ORM****:* ORM frameworks like Hibernate automate many of the tedious aspects of database management, such as writing SQL queries and managing connections, which can lead to faster development and more maintainable code.

Limitations of JDBC (Java Database Connectivity)

1. **SQL Query Language Dependency**:
    - Developers must manually write and manage SQL queries for all database operations.
    - This increases the complexity and potential for errors, especially in large applications with complex queries.
2. **Repetitive Code**:
    - JDBC requires boilerplate code for tasks such as establishing connections, creating statements, and handling exceptions.
    - The repetitive nature of this code can lead to increased development time and maintenance challenges.
3. **Lack of Named Parameters**:
    - JDBC only supports positional parameters in prepared statements (e.g., ? placeholders), making it harder to manage queries with multiple parameters.
    - Named parameters, which allow parameters to be identified by name rather than position, are not supported, reducing the readability and flexibility of queries.

These limitations often lead developers to seek out higher-level frameworks like Hibernate, which abstract away much of the complexity involved in database interactions and provide more features such as named parameters and automatic SQL generation.

JPA and Hibernate as ORM Framework



What is ORM (Object-Relational Mapping)?

- ***Definition****:* ORM is a technique that allows developers to map Java objects to database tables, making it easier to work with databases in an object-oriented manner.
- ***How It Works****:*
    - Java classes (representing entities) are mapped to database tables.
    - Each instance of a class corresponds to a row in the table.
    - This mapping allows developers to manipulate database records as if they were working with simple Java objects.

JPA (Java Persistence API):

- ***Definition:*** JPA is a specification provided by Java for managing relational data in Java applications.
- ***Key Points:***
    - ***Object-Relational Mapping:*** It defines how to map Java objects to database tables.
    - ***Implementations:*** Hibernate and TopLink are popular implementations of the JPA specification.

Hibernate:

- ***Definition:*** Hibernate is a popular ORM framework that implements the JPA specification, providing a powerful and flexible way to work with relational databases in Java.
- ***How It Works:***
    - Hibernate automatically generates SQL queries based on the mapping defined between Java classes and database tables.
    - These queries are then executed using JDBC (Java Database Connectivity) to interact with the database.

Key Benefits:

- ***Automated SQL Generation:*** Hibernate generates SQL queries automatically, reducing the need for manual query writing.
- ***Ease of Data Manipulation:*** Developers can work with database data as Java objects, simplifying code and improving maintainability.

Summary:

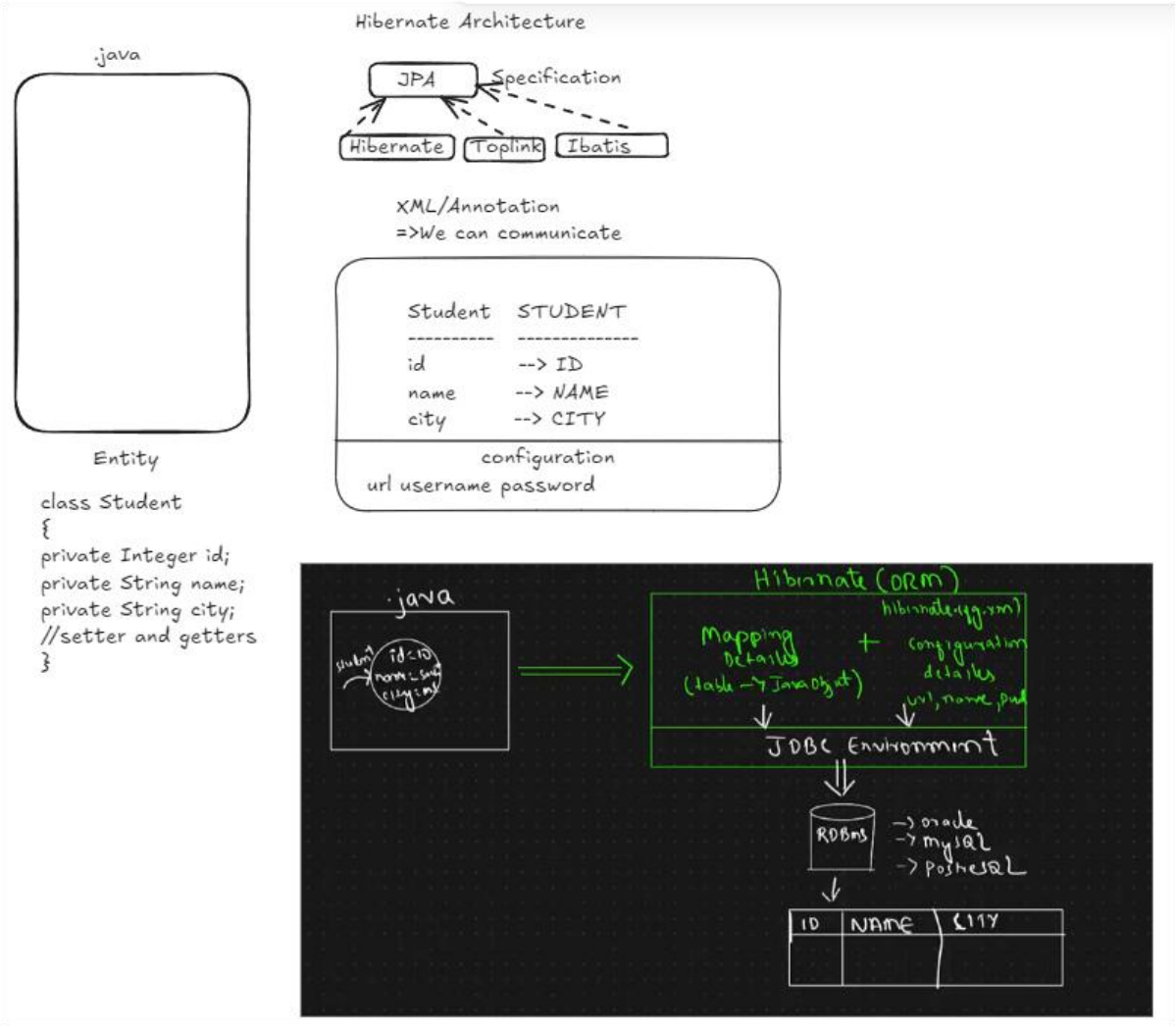
- ***ORM (Object-Relational Mapping):*** Maps Java classes to database tables.
- ***JPA (Java Persistence API):*** A specification that defines how ORM should be implemented in Java.

Hibernate: A widely-used implementation of JPA that automates database interactions, making Java development more efficient and less error-prone.

Hibernate Architecture

Hibernate is a powerful, high-performance Object-Relational Mapping (ORM) tool that simplifies the interaction between Java applications and relational databases. It abstracts the database operations by allowing developers to focus on the business logic rather than SQL queries.

Hibernate Architecture image



Core Components of Hibernate Architecture:

1. **JPA (Java Persistence API) Specification:**
    - Hibernate implements the JPA specification, which standardizes ORM in Java.

- It provides a common interface for different ORM tools like Hibernate, Toplink, and iBatis.

2. **Mapping Configuration:**
    - Hibernate uses XML files or annotations to map Java objects to database tables.
    - Each Java class that needs to be persisted in the database is known as an **Entity**.

```
class Student {
   private Integer id;
   private String name;
   private String city;
   // Getters and setters
}
```

The class Student is mapped to the STUDENT table in the database with fields id, name, and city corresponding to the columns ID, NAME, and CITY.

Hibernate Configuration:

- Hibernate configuration involves setting up the database connection details such as the URL, username, and password.
- This configuration can be done through an XML file (hibernate.cfg.xml) or programmatically via annotations.

Hibernate ORM (Object-Relational Mapping):

- Hibernate converts Java objects into database records (rows in tables) and vice versa.
- The process of converting a Java object into a database record is called **Mapping**. This involves:

        - **Mapping Details:** Java objects are mapped to relational database tables.
        - **Configuration Details:** These include the database connection properties and other settings needed for the ORM tool to function correctly.
    - Hibernate interacts with the **JDBC Environment** to execute SQL queries and manage database transactions.

Database Connectivity:

- Hibernate can work with various relational databases (RDBMS) like Oracle, MySQL, PostgreSQL, etc.
- The ORM tool abstracts the underlying database operations, allowing developers to switch between different databases with minimal configuration changes.

Entity & Table Mapping:

- The entities (e.g., Student) in Hibernate are directly mapped to corresponding database tables (STUDENT).
- Each field in the entity class (e.g., id, name, city) is mapped to a column in the database table.

Process Flow in Hibernate:

1. **Java Object Creation:**
    - A Java object (e.g., Student) is created with data.
2. **ORM Handling:**
    - Hibernate ORM handles the conversion of this object into a database record by using the mapping configuration.
3. **Database Operation:**
    - Hibernate interacts with the JDBC environment to insert, update, or query the database.

- The data is stored in the appropriate table (e.g., STUDENT table) in the chosen RDBMS.

Hibernate Project Setup

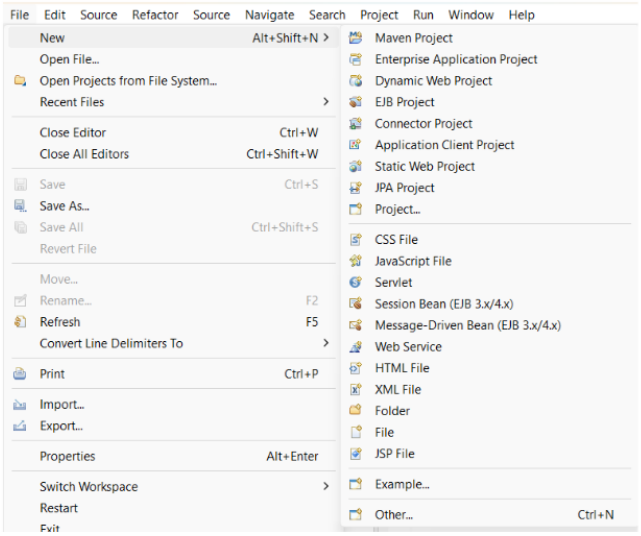
What have we learned?

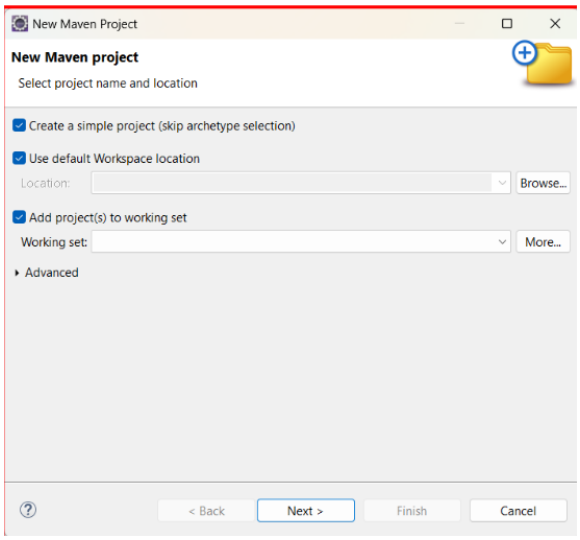
. How to set up a hibernate project?

. How to change the default JRE library?

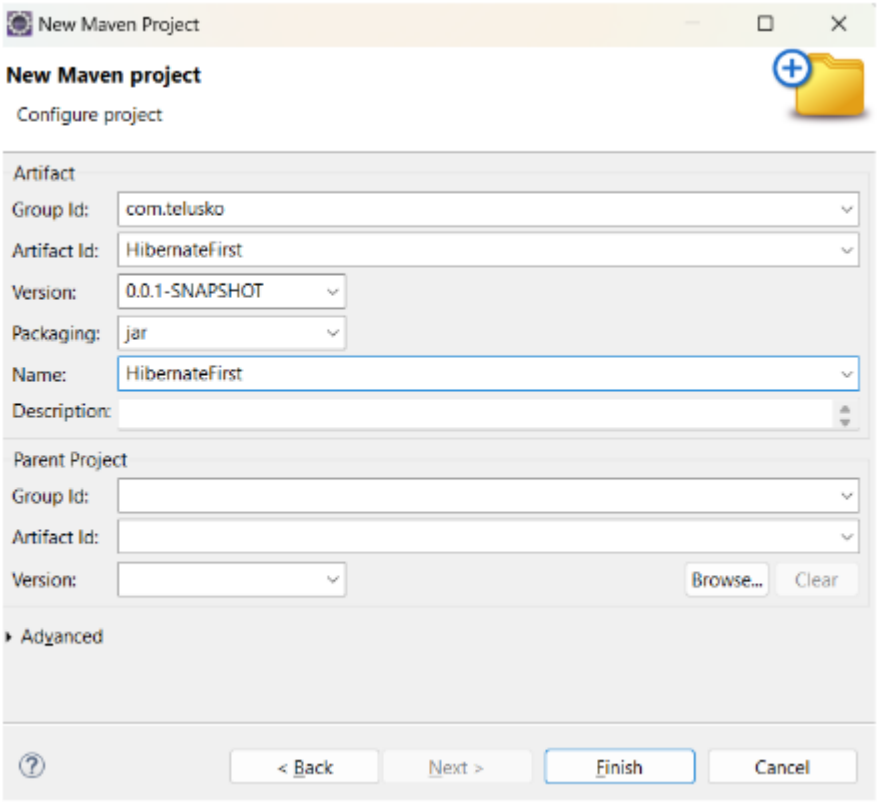
Steps to Create a Hibernate Project

- **Step 1**: Create a Maven project using the quickstart template.

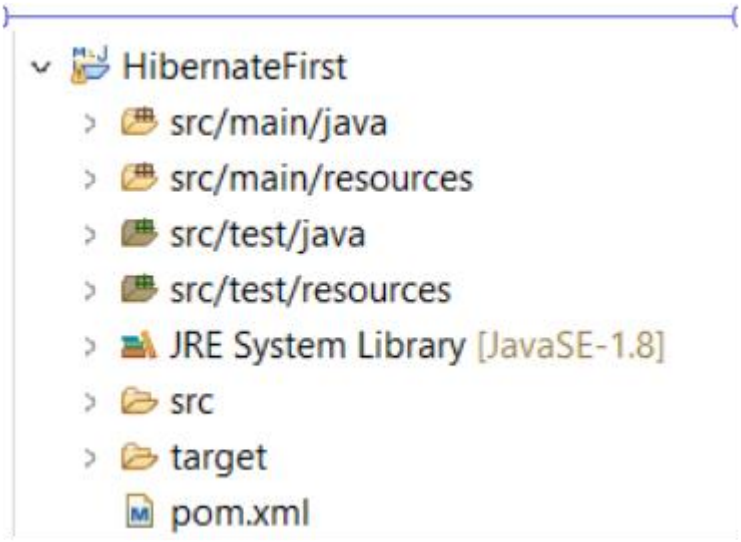




- **Step 2**: Add the Group ID, Artifact ID, and Project Name.



- **Step 3**: Open pom.xml and add the hibernate-core dependency.



Add dependency:

```
<!--
https://mvnrepository.com/artifact/org.hibernate/hibernate-
core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.5.2.Final</version>
    <type>pom</type>
</dependency>
```

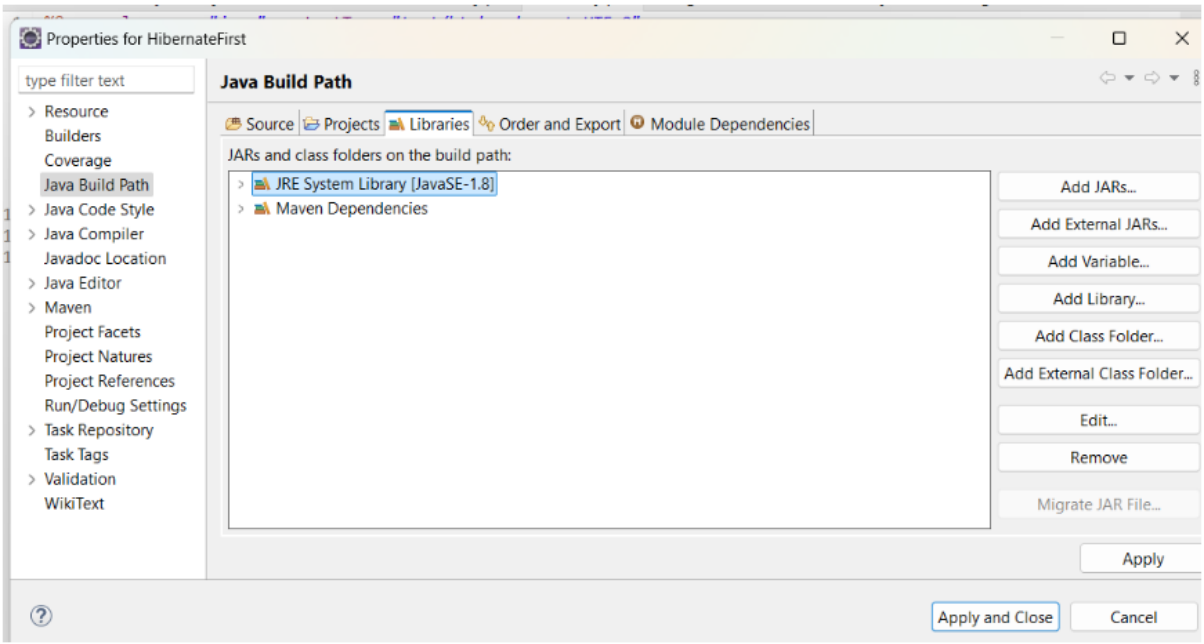
pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.telusko</groupId>
  <artifactId>HibernateFirst</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>HibernateFirst</name>

  <dependencies>
  <!--
https://mvnrepository.com/artifact/org.hibernate/hibernate-
core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
```

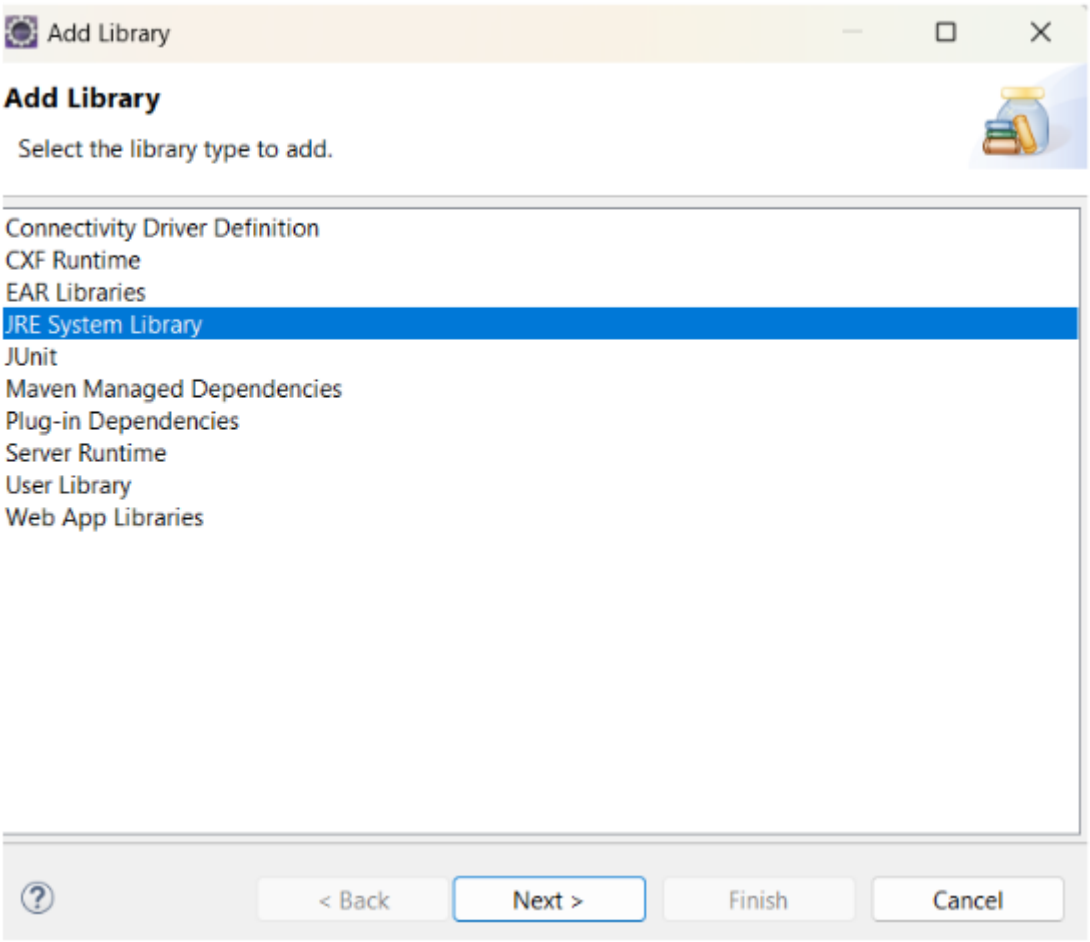
```
    <version>6.5.2.Final</version>
    <type>pom</type>
</dependency>
  </dependencies>
</project>
```

Changing the Default JRE Library

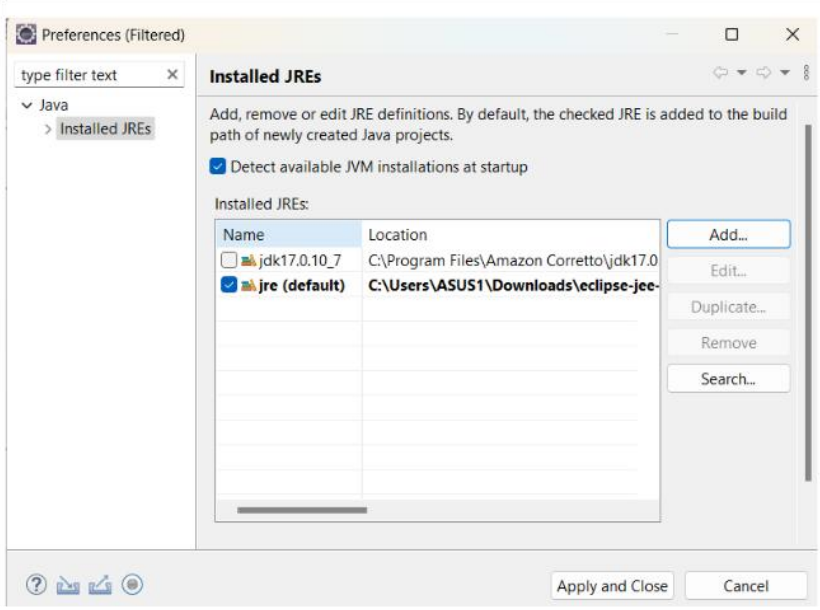
- **Step 1**: Open the Java build path settings.

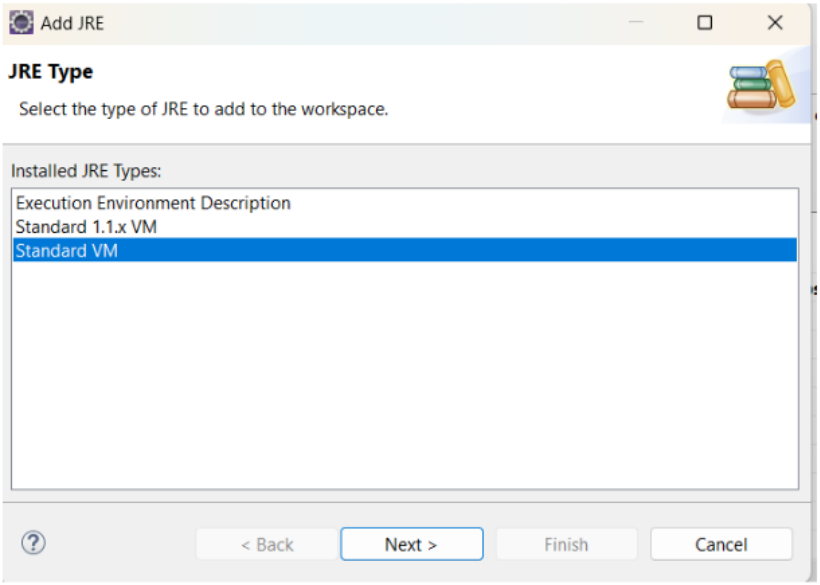


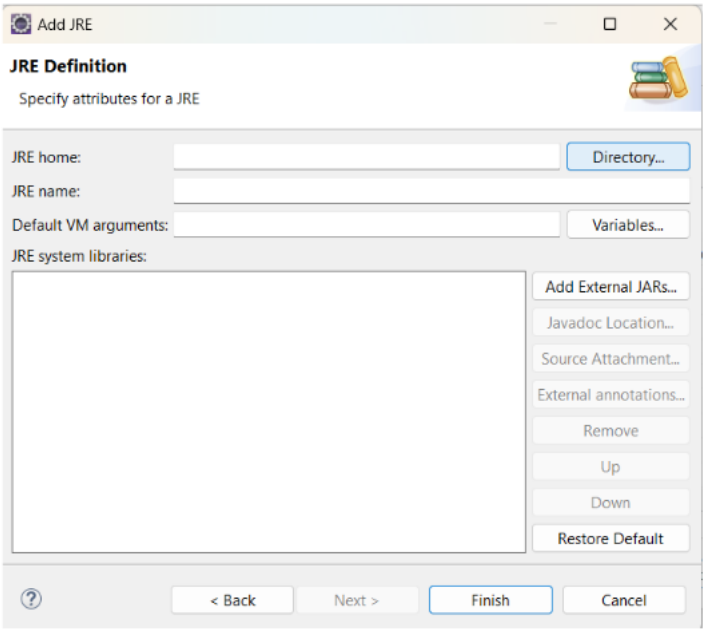
- **Step 2**: Remove the default JRE library.
- **Step 3**: Click "Add Library" and select "JRE System Library."

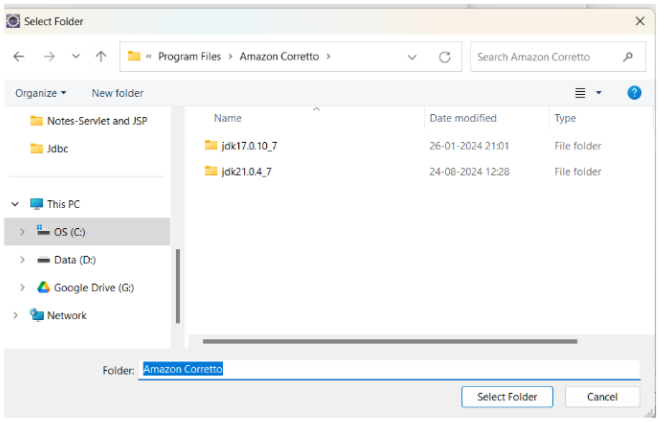


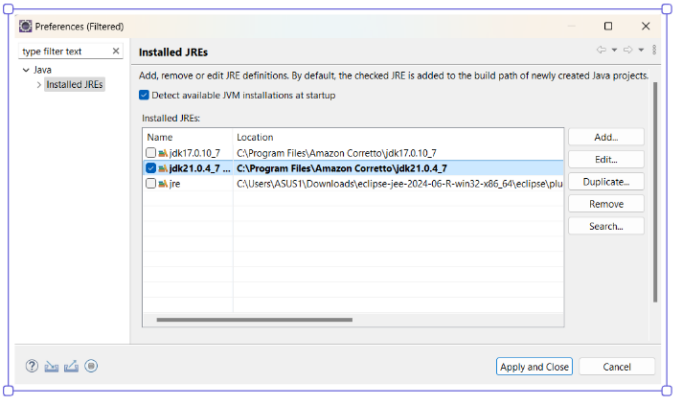
- **Step 4**: Choose the JDK directory where your installed JDK is located (e.g., JDK 21 or 17).


Preferences (Filtered)
type filter text
Java
Installed JREs
Installed JREs
Add, remove or edit JRE definitions. By default, the checked JRE is added to the build path of newly created Java projects.
Detect available JVM installations at startup
Installed JREs:
Name
Location
jdk17.0.10_7
C:\Program Files\Amazon Corretto\jdk17.0
jre (default)
C:\Users\ASUS1\Downloads\eclipse-jee-
Add...
Edit...
Duplicate...
Remove
Search...
Apply and Close
Cancel

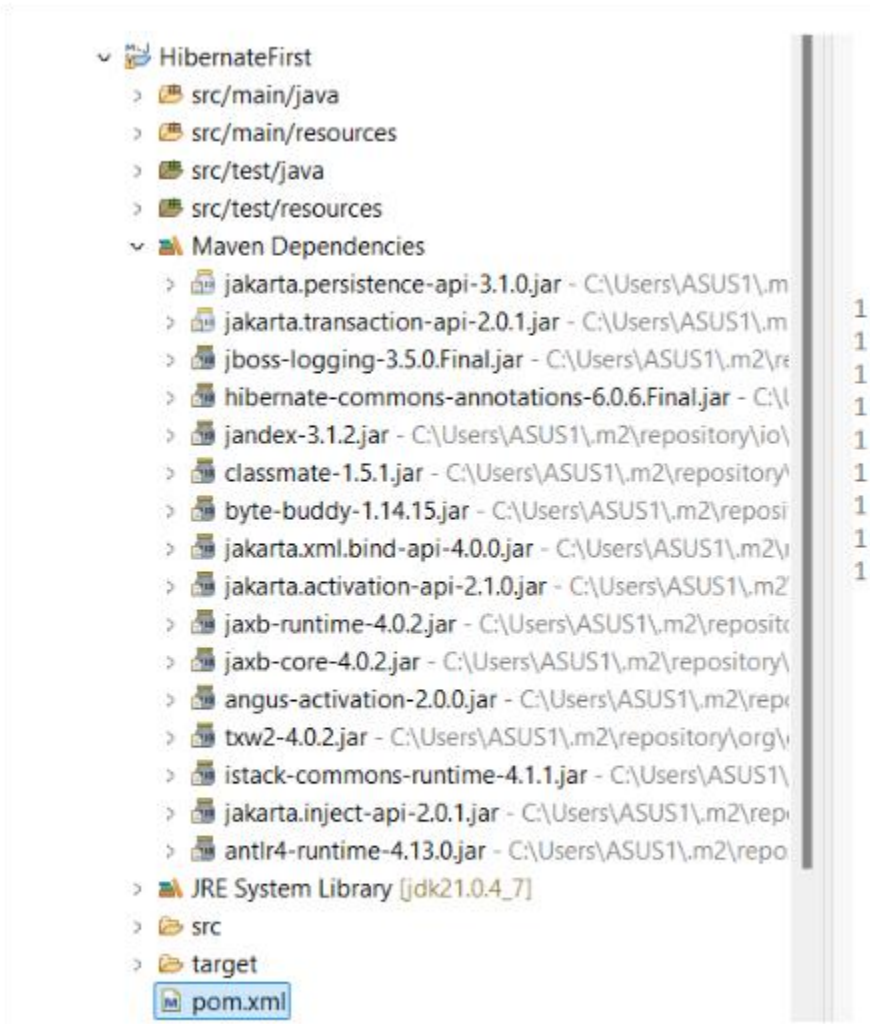

Add JRE
JRE Type
Select the type of JRE to add to the workspace.
Installed JRE Types:
Execution Environment Description
Standard 1.1.x VM
Standard VM
< Back
Next >
Finish
Cancel



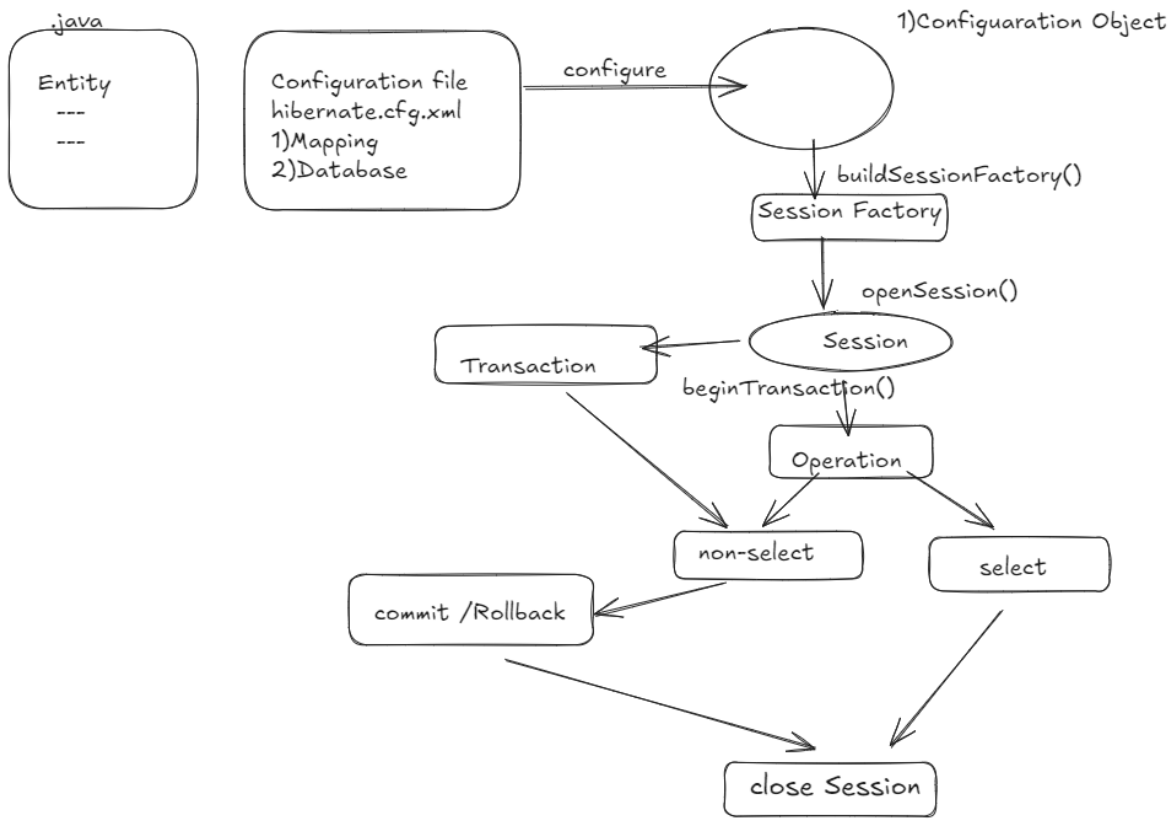




- **Step 5**: Apply the changes and finish.


HibernateFirst
src/main/java
src/main/resources
src/test/java
src/test/resources
Maven Dependencies
jakarta.persistence-api-3.1.0.jar - C:\Users\ASUS1\.m
jakarta.transaction-api-2.0.1.jar - C:\Users\ASUS1\.m
jboss-logging-3.5.0.Final.jar - C:\Users\ASUS1\.m2\re
hibernate-commons-annotations-6.0.6.Final.jar - C:\
jandex-3.1.2.jar - C:\Users\ASUS1\.m2\repository\io\
classmate-1.5.1.jar - C:\Users\ASUS1\.m2\repository\
byte-buddy-1.14.15.jar - C:\Users\ASUS1\.m2\reposi
jakarta.xml.bind-api-4.0.0.jar - C:\Users\ASUS1\.m2\
jakarta.activation-api-2.1.0.jar - C:\Users\ASUS1\.m2
jaxb-runtime-4.0.2.jar - C:\Users\ASUS1\.m2\reposit
jaxb-core-4.0.2.jar - C:\Users\ASUS1\.m2\repository\
angus-activation-2.0.0.jar - C:\Users\ASUS1\.m2\rep
txw2-4.0.2.jar - C:\Users\ASUS1\.m2\repository\org\
istack-commons-runtime-4.1.1.jar - C:\Users\ASUS1\
jakarta.inject-api-2.0.1.jar - C:\Users\ASUS1\.m2\rep
antlr4-runtime-4.13.0.jar - C:\Users\ASUS1\.m2\repo
JRE System Library [jdk21.0.4_7]
src
target
pom.xml
1
1
1
1
1
1
1
1
1

Hibernate Project Setup Architecture



Note: No transaction needed for select operation.

1. **Entity Class (Java):**
    - Represents the Java class that needs to be persisted in the database.
    - This class is mapped to a database table via configuration.
2. **Configuration File (hibernate.cfg.xml):**
    - The central configuration file for Hibernate.
    - It includes details for:
        - **Mapping:** Specifies how Java classes are mapped to database tables.
        - **Database Configuration:** Contains information like the database URL, username, and password.
3. **Configuration Object:**

- Loads the Hibernate configuration file.
- It initializes Hibernate using the configure() method.

4. SessionFactory:

- Created by the buildSessionFactory() method.
- It is a factory for Session objects.
- Designed to be instantiated once and used throughout the application lifecycle.

5. Session:

- The primary interface for interacting with the database.
- Obtained from the SessionFactory using openSession().
- Used to create transactions and perform database operations.

6. Transaction:

- Represents a unit of work in Hibernate.
- Managed through the beginTransaction() method.
- Operations can be either:
  - **Non-Select Operations:** Involve insert, update, or delete actions.
  - **Select Operations:** Involve querying the database to retrieve data. (Note: does not required for select query)

7. Commit/Rollback:

- After the operations are performed, the transaction must be either committed or rolled back.
- **Commit:** Saves the changes to the database.
- **Rollback:** Reverts the changes in case of an error or cancellation.

8. Close Session:

- The session should be closed after completing the operations.
- This releases the database connection and other resources.

Install the Helper plugin in Eclipse IDE

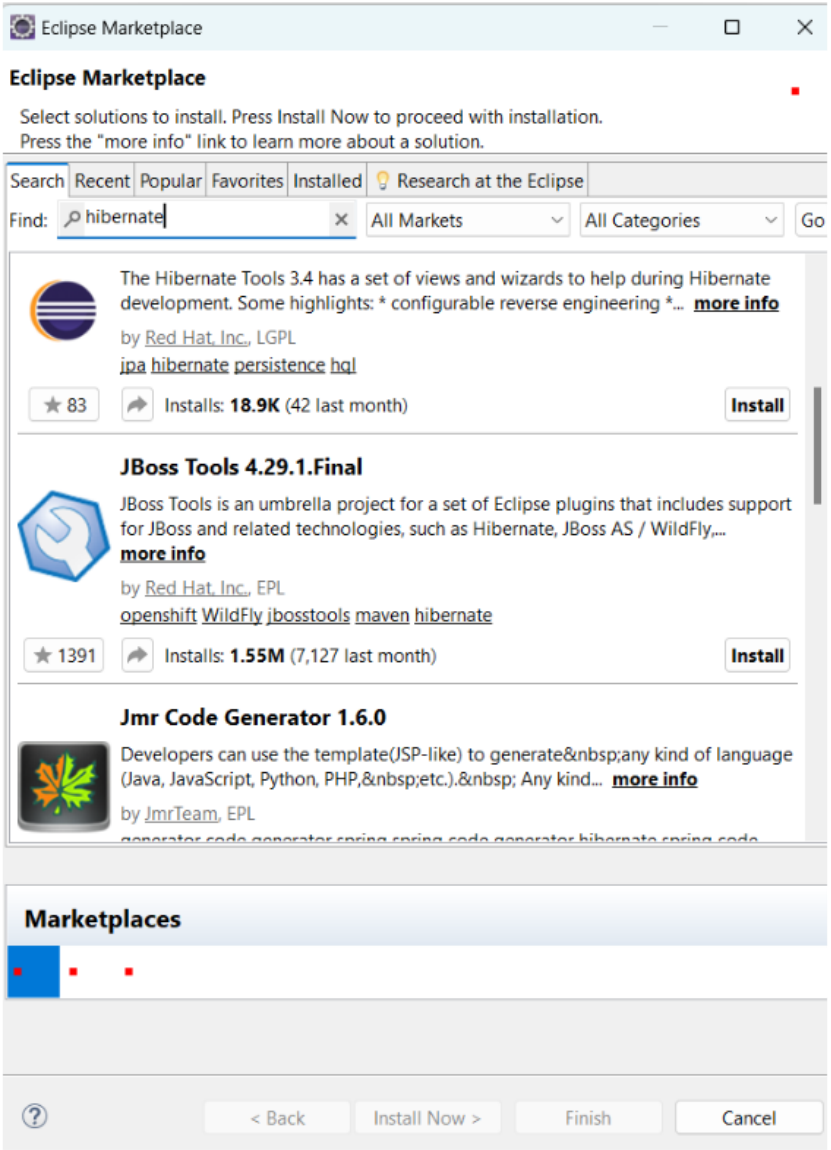
Need of JBOSS plugin (specific for Eclipse IDE)

The JBoss Tools plugin for Hibernate simplifies development by generating configuration files, entity classes, and CRUD operations automatically. It includes an HQL editor, schema visualization, and seamless Eclipse integration, saving time and enhancing productivity.

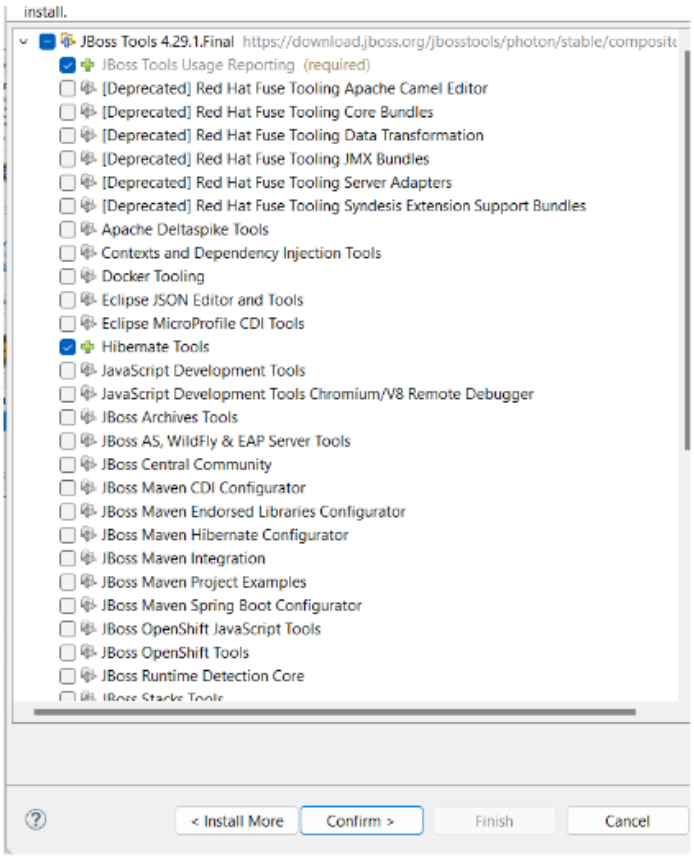
Steps to install the helper plugin (JBOSS) for Hibernate:

Step1: Click on Help and then select Eclipse Marketplace.

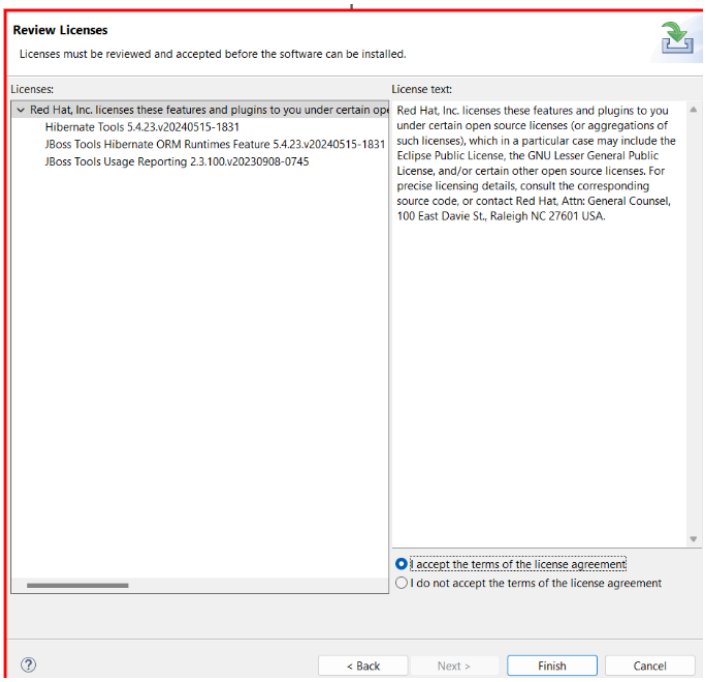
Choose Jboss



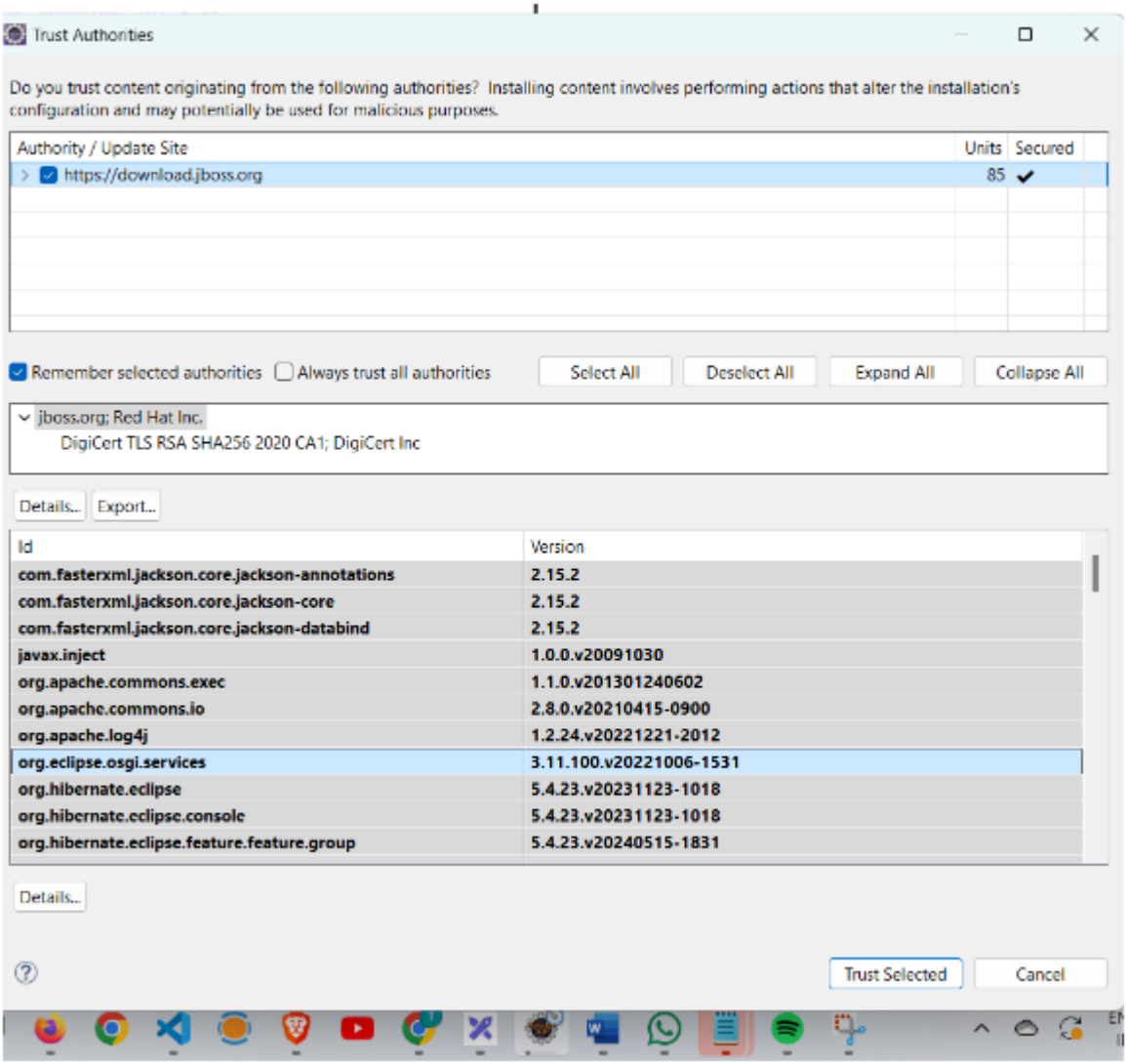
Step 2: Select only Hibernate tools.



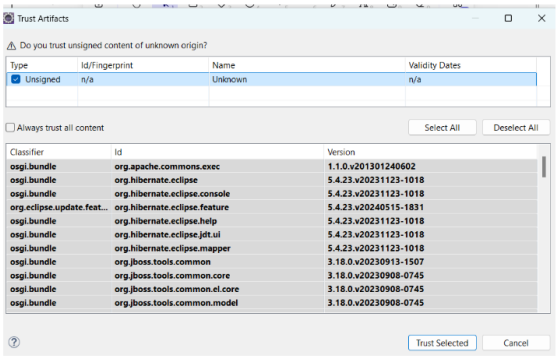
Step3: Accept license



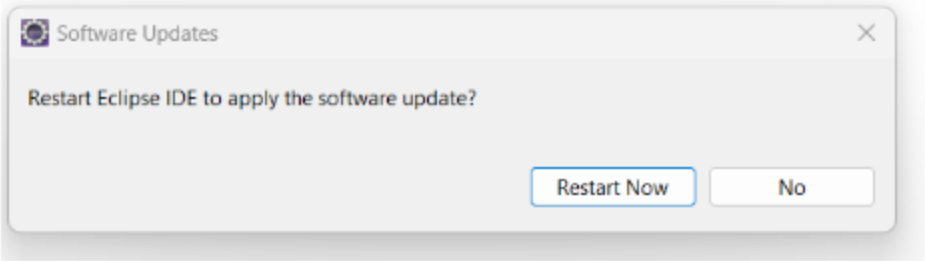
Step(optional): If you get such things, just **check the** check option and then click on **Trust selected.**



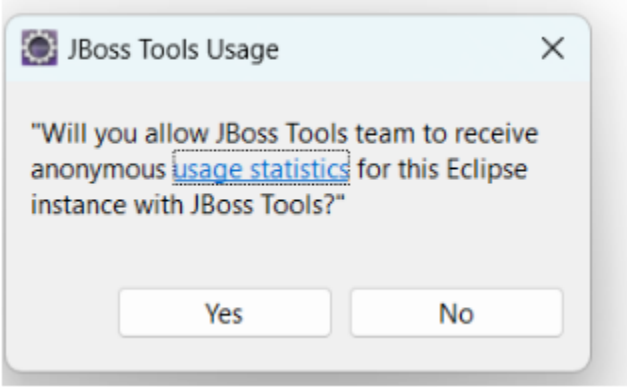
Step(Optional): If you get such things, just **check the** check option and then click on **Trust selected.**



Step(final): Restart your eclipse



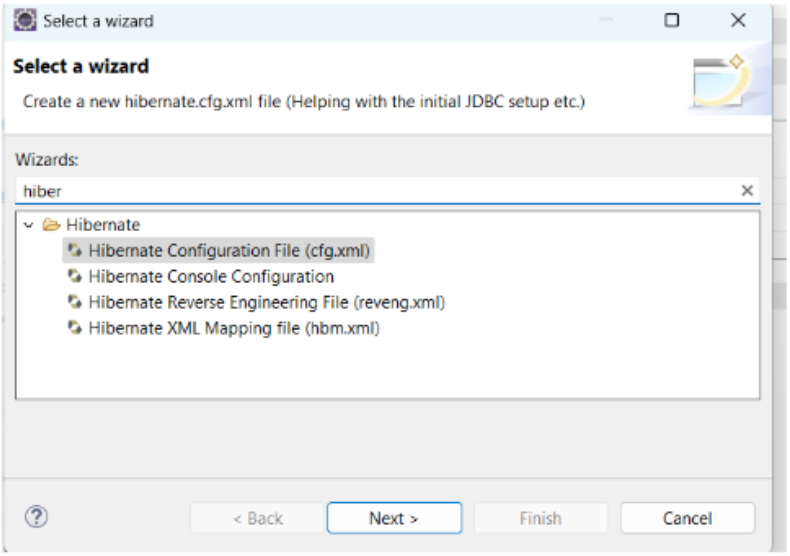
Step(Optional): If you get after restarting then select Yes.

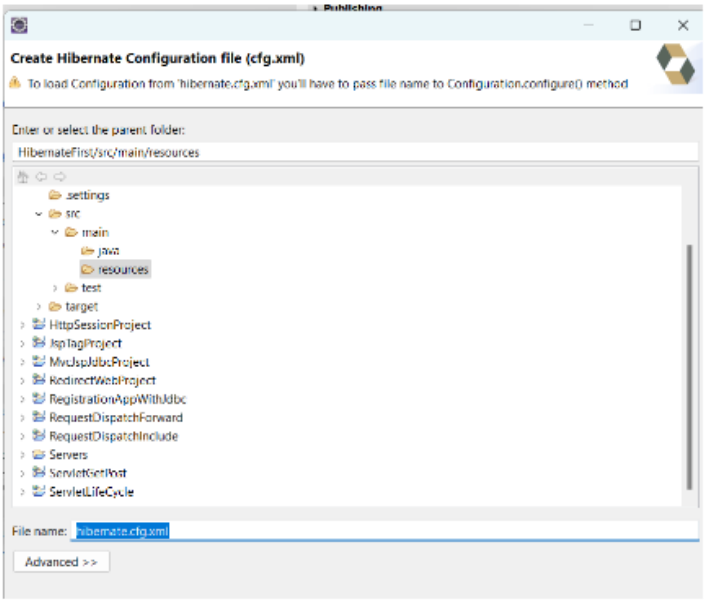


Create a hibernate.cfg.xml file using a plugin.

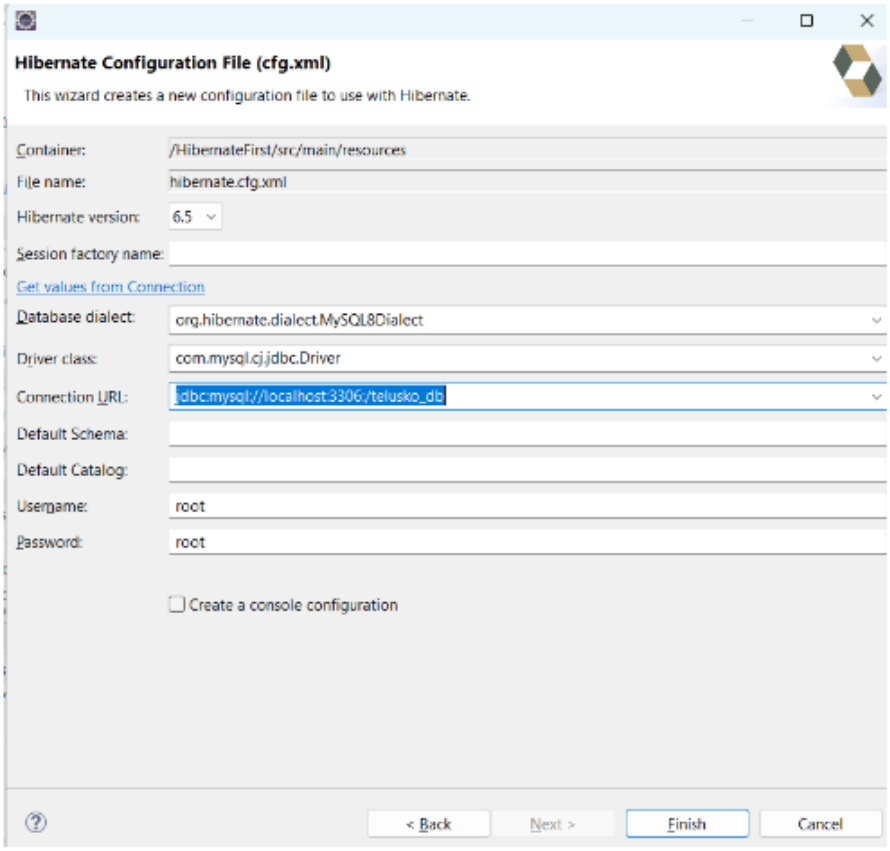
Step1: Search hibernate and choose cfg.xml file

Create the name of the XML file by (hibernate.cfg.xml)

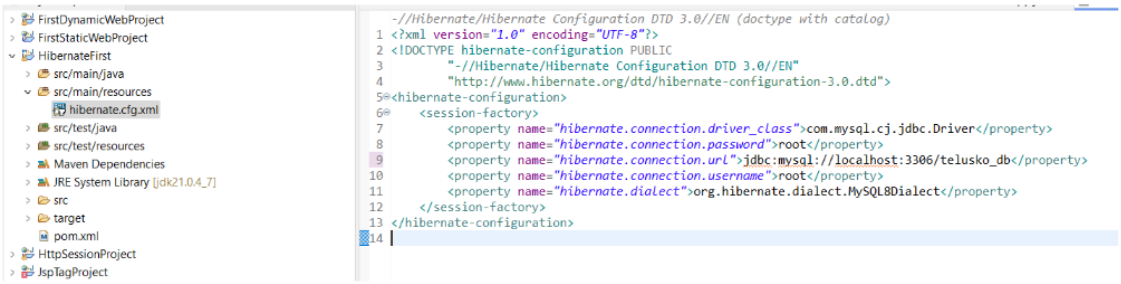




Step2: Enter details of configurations



Final file:



First Hibernate Application

Hibernate is a popular ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java classes to database tables. Below are the key components and steps for creating your first Hibernate application.

1. Model Class: Student

- **Annotations Used:**
  - @Entity: Marks the class as an entity, meaning it will be mapped to a database table.
  - @Table(name = "StudentTable"): Specifies the name of the database table.
  - @Id: Marks the primary key of the entity.
  - @Column(name = "SID"): Maps the field to the corresponding column in the database.
- **Code Example:**

```
@Entity
@Table(name="StudentTable")
public class Student {

	@Id
	@Column(name="SID")
	private Integer sid;

	@Column(name="SNAME")
	private String sname;

	@Column(name="SCITY")
	private String scity;

	public Student() {
	System.out.println("Zero Param Constructor for Hibernate");
```

```
    }

    // Getters and Setters
}
```

Notes: If you do not specify the table name or column names in Hibernate using the @Table and @Column annotations, Hibernate will apply default naming conventions:

1. **Default Table Name:**
    - By default, the table name will be the same as the class name of the entity.
    - For example, if your entity class is named Student, the default table name will be Student.
2. **Default Column Names:**
    - The column names will be the same as the field names in your entity class.
    - For example, if you have a field private String sname;, the default column name will be sname.

2. Hibernate Configuration File (hibernate.cfg.xml)

- **Important Properties:**
    - hibernate.connection.driver_class: Specifies the JDBC driver class.
    - hibernate.connection.url: The JDBC URL to connect to the database.
    - hibernate.connection.username and hibernate.connection.password: Credentials for the database.
    - hibernate.dialect: Specifies the SQL dialect Hibernate should use (e.g., org.hibernate.dialect.MySQL8Dialect).
    - hibernate.hbm2ddl.auto: Specifies how Hibernate should handle database schema creation/update. Values can be create, update, validate, etc.
    - hibernate.show_sql: If set to true, Hibernate will log the generated SQL to the console.

    - hibernate.format_sql: Formats the SQL output in the console for better readability.
    - <mapping class="com.telusko.model.Student"/>: Specifies the class to be mapped to the database.
- **Code Example:**

```
<hibernate-configuration>
	<session-factory>
	<property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
	<property name="hibernate.connection.url">jdbc:mysql://localhost:3306/yourDatabaseName</property>
	<property name="hibernate.connection.username">root</property>
	<property name="hibernate.connection.password">root123</property>
	<property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
	<property name="hibernate.hbm2ddl.auto">create</property>
	<property name="hibernate.show_sql">true</property>
	<property name="hibernate.format_sql">true</property>
	<mapping class="com.telusko.model.Student"/>
	</session-factory>
</hibernate-configuration>
```

3. Main Class for Hibernate Operations

- **Steps Involved:**
    1. **Configuration Object**: Loads the Hibernate configuration file.
    2. **Build SessionFactory**: Creates a SessionFactory from the configuration object.
    3. **Open Session**: Opens a new session from the SessionFactory.
    4. **Begin Transaction**: Begins a transaction for database operations.

5. **Perform Operations**: In this example, create a Student object, set its properties, and save it to the database.
6. **Commit Transaction**: Commits the transaction to persist the changes.
7. **Close Session**: Closes the session after the operation is complete.

- **Code Example:**

```
public static void main(String[] args) {
        // Step 1: Configuration Object
        Configuration config = new Configuration();
    config.configure();

        // Step 2: Create SessionFactory Object
        SessionFactory sessionFactory = config.buildSessionFactory();

        // Step 3: Get the Session Object from Session Factory
        Session session = sessionFactory.openSession();

        // Step 4: Begin the Transaction within Session
        Transaction transaction = session.beginTransaction();

        // Step 5: Perform Operation
        Student student = new Student();
        student.setSid(1233);
        student.setSname("Arjun");
        student.setScity("Ballia");

    session.save(student);

        // Step 6: Commit Transaction
    transaction.commit();

        // Step 7: Close the Session
        session.close();
```

```
}
```

Note on Configuring hibernate.cfg.xml File

- **Default Behavior:** When configuring Hibernate, the framework automatically looks for a configuration file named hibernate.cfg.xml in the classpath. You can load this configuration using:

```
Configuration config = new Configuration();
config.configure();
```

- **Custom Configuration File Name:**
  - If your configuration file is named something other than hibernate.cfg.xml (e.g., hib.cfg.xml), you must explicitly specify this file name when configuring Hibernate.
  - Example:

```
Configuration config = new Configuration();
config.configure("hib.cfg.xml");
```

Key Points:

- **Default:** config.configure(); assumes the file is named hibernate.cfg.xml.
- **Custom:** Use config.configure("custom-file-name.xml"); if you have renamed your configuration file.

Output:

```
mysql> use telusko_db
Database changed
mysql> select * from Student;
ERROR 1146 (42S02): Table 'telusko_db.student' doesn't exist
mysql> show tables;
+----------------------+
| Tables_in_telusko_db |
+----------------------+
| personinfo           |
| studenttable         |
+----------------------+
2 rows in set (0.01 sec)

mysql> select * from studenttable;
+------+--------+-------+
| sid  | scity  | sname |
+------+--------+-------+
|  123 | Ballia | Shiva |
| 1233 | Ballia | Arjun |
+------+--------+-------+
2 rows in set (0.00 sec)
```

Insert Using persist() method

In Hibernate, inserting data into the database is typically achieved using session operations like persist(), save(), or saveOrUpdate(). This process involves creating an entity, starting a transaction, and using the session object to persist the entity into the database.

Steps to Insert Data Using Hibernate:

1. **Configure Hibernate:**
    - Create a Configuration object to configure Hibernate with properties and mappings.
    - Load the hibernate.cfg.xml file with config.configure().
2. **Create a SessionFactory and Session:**
    - SessionFactory is a factory for creating Session objects.
    - Open a session using sessionFactory.openSession().
3. **Begin a Transaction:**
    - Always begin a transaction before performing any database operation.
    - Use session.beginTransaction() to start a transaction.
4. **Create an Entity Object:**
    - Instantiate the entity object (e.g., Student) and set its properties.
5. **Insert Entity Using session.persist():**
    - Use session.persist() to insert the entity into the database.
    - Commit the transaction using transaction.commit().
6. **Handle Exceptions:**
    - Catch exceptions such as EntityExistsException, ConstraintViolationException, and HibernateException.
7. **Close Resources:**
    - Use session.close() and sessionFactory.close() in the finally block.

Exceptions during Insert Operations in Hibernate

1. **EntityExistsException:**
    - Thrown when trying to persist an entity that already exists in the database.

```
catch (EntityExistsException e) {
   System.out.println("Entity already exists: " + e.getMessage());
}
```

2. **ConstraintViolationException:**
    - Thrown when database integrity constraints (like unique constraints) are violated.

```
catch (ConstraintViolationException e) {
   System.out.println("Constraint violation: " + e.getMessage());
}
```

3. **HibernateException:**
    - A general exception that indicates a problem with the Hibernate framework.

```
catch (HibernateException e) {
   System.out.println("Hibernate exception: " + e.getMessage());
}
```

4. **Exception:**
    - Catches any other unexpected exceptions.

```
catch (Exception e) {
 System.out.println("Unexpected exception: " + e.getMessage());
}
```

persist() vs save() in Hibernate

1. **persist() Method:**
    - **Purpose:** Used to insert an entity into the database if it does not already exist.
    - **Return Type:** void (does not return the identifier).
    - **Behavior:**

        - Makes the given entity persistent and schedules it for insertion during flush/commit.
        - Does **not** guarantee the immediate insertion.
    - **Exception Handling:** Throws EntityExistsException if the entity already exists in the database.

```
session.persist(student);
```

2. **save() Method:**
    - **Purpose:** Used to insert an entity into the database.
    - **Return Type:** Returns the generated identifier (primary key) of the entity.
    - **Behavior:**
        - Guarantees the immediate insertion of the entity.
        - The generated identifier is immediately available after the call.
    - **Exception Handling:** Unlike persist(), save() will insert the entity, even if it is in the transient state.

```
Serializable id = session.save(student);
System.out.println("Student ID: " + id);
```

Example of save() and persist() Methods in Hibernate

Using persist():

```
Transaction transaction = session.beginTransaction();
Student student = new Student();
student.setId(134);
student.setName("Arjun");
student.setCity("Ballia");

session.persist(student);  // Insert the student entity
transaction.commit();      // Commit the transaction
```

Using save():

```
Transaction transaction = session.beginTransaction();
Student student = new Student();
student.setName("Arjun");
student.setCity("Ballia");

Serializable id = session.save(student);  // Insert the student entity and return the ID
transaction.commit();
System.out.println("Student ID: " + id);  // Print the generated ID
```

Key Differences Between persist() and save():

Aspect	persist()	save()
Return Type	void	Returns Serializable (primary key ID)
Entity State	Transient -> Persistent	Transient -> Persistent
Insert Guarantee	Not immediate, scheduled at flush	Immediate upon calling
Exception Handling	Throws EntityExistsException	No exception for existing entities
Recommended Usage	For standardized JPA usage	When you need the generated ID

Industry Standard Codes:

Main.java

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
  public static void main(String[] args) {
      Configuration config = null;
      SessionFactory sessionFactory = null;
      Session session = null;
      Transaction transaction = null;
      boolean flag = false;
      try {
          config = new Configuration();
          config.configure();
          sessionFactory = config.buildSessionFactory();
          session = sessionFactory.openSession();
          Student student = new Student();
          student.setId(134);
          student.setName("Arjun");
          student.setCity("Ballia");
          transaction = session.beginTransaction();
          session.persist(student);
          transaction.commit();
          System.out.println("Transaction commited Succesfully");
          flag = true;
      } catch (EntityExistsException e) {
          System.out.println("Entity already exists: " + e.getMessage());
      } catch (ConstraintViolationException e) {
```

```
            System.out.println("Constraint violation: " + e.getMessage());
        } catch (HibernateException e) {
            System.out.println("Hibernate exception: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected exception: " + e.getMessage());
        } finally {
          if(!flag)
          {
              if (transaction != null) {
                  transaction.rollback();
                  System.out.println("Transaction rolled back successfully");
              }
          }
          if (session != null) {
              session.close();
          }
          if (sessionFactory != null) {
              sessionFactory.close();
          }
        }
    }
}
```

Student.java

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="StudentTable")
public class Student {
    @Id
```

```
    @Column(name="sid")
        private int id;

    @Column(name="sname")
        private String name;

    @Column(name="scity")
        private String city;


        public Student() {
                System.out.println("Zero param constructor");
        }

        public int getId() {
                return id;
        }
        public void setId(int id) {
                this.id = id;
        }
        public String getName() {
                return name;
        }
        public void setName(String name) {
                this.name = name;
        }
        public String getCity() {
                return city;
        }
        public void setCity(String city) {
                this.city = city;
        }

}
```

Result:

```
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionPr
Zero param constructor
Hibernate:
    insert
    into
        StudentTable
        (scity, sname, sid)
    values
        (?, ?, ?)
Transaction commited Succesfully
Sept 07, 2024 3:36:55 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/telusko_db]
```

```
mysql> select *  from studenttable;
+------+--------+-------+
| sid  | scity  | sname |
+------+--------+-------+
|   13 | Ballia | Arjun |
|  123 | Ballia | Shiva |
|  134 | Ballia | Arjun |
| 1233 | Ballia | Arjun |
| 1345 | Ballia | Shiva |
+------+--------+-------+
5 rows in set (0.00 sec)
```

Final Notes:

- Always handle exceptions gracefully to ensure the transaction is rolled back when necessary.
- Use persist() when following JPA specifications and save() when you need to get the generated identifier immediately.

Updating the Data

1. **One-Line Configuration using addAnnotatedClass()**:
    - Replaces the need for hibernate.cfg.xml.

```
 SessionFactory sessionFactory = new Configuration().configure().addAnnotatedClass(Student.class).buildSessionFactory();
```

Overall Workflow:

1. new Configuration(): Creates a configuration object.
2. configure(): Loads the Hibernate configuration details.
3. addAnnotatedClass(Student.class): Registers the Student class as an entity.
4. buildSessionFactory(): Builds the SessionFactory, which is used to create sessions for interacting with the database.

2. **update()**:
    - Used to update an existing record in the database.
    - The entity must already exist in the database (identified by its ID).
    - Throws an exception if the entity is not already in the database.
    - Example:

```
session.update(student);
```

3. **saveOrUpdate()**:
    - Saves a new entity or updates an existing one based on its identifier (ID).
    - If the entity is new (no ID in the database), it inserts. If it exists, it updates.
    - Example:

```
session.saveOrUpdate(student);
```

4. **merge()**:
    - Similar to saveOrUpdate(), but works with detached entities (not in session).
    - Returns a new instance that is managed by Hibernate; does not directly update the original object.
    - Useful when working with multiple sessions.
    - Example:

```
Student updatedStudent = session.merge(student);
```

Key Differences:

- **save()**: Only inserts new records. Throws an exception if the record exists.
- **saveOrUpdate()**: Inserts if new, updates if the record already exists.
- **merge()**: Updates detached entities and works with a new instance, does not modify the original entity directly.

Code:

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
  public static void main(String[] args) {
     Configuration config = null;
     SessionFactory sessionFactory = null;
     Session session = null;
     Transaction transaction = null;
     boolean flag = false;
     try {
        config = new Configuration();
```

```
            config.configure();
            sessionFactory = config.buildSessionFactory();
            session = sessionFactory.openSession();
            // Detached entity that we want to update in the database
            Student student = new Student();
            student.setId(134588); // Make sure this ID exists in the database for an
update
            student.setName("Shiva Updated with Merge");
            student.setCity("New City Ballia");
            transaction = session.beginTransaction();
//            session.update(student);
//            session.saveOrUpdate(student);
            Student updatedStudent =   session.merge(student);  // Merge operation
            transaction.commit();
            System.out.println("Transaction committed successfully using merge.");
            flag = true;
        } catch (EntityExistsException e) {
            System.out.println("Entity already exists: " + e.getMessage());
        } catch (ConstraintViolationException e) {
            System.out.println("Constraint violation: " + e.getMessage());
        } catch (HibernateException e) {
            System.out.println("Hibernate exception: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected exception: " + e.getMessage());
        } finally {
            if (!flag) {
                if (transaction != null) {
                    transaction.rollback();
                    System.out.println("Transaction rolled back successfully");
                }
            }
            if (session != null) {
                session.close();
            }
```

```
        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
  }
}
```

Result:

```
Zero param constructor
Hibernate:
    select
        s1_0.sid,
        s1_0.scity,
        s1_0.sname
    from
        StudentTable s1_0
    where
        s1_0.sid=?
Zero param constructor
Transaction committed successfully using merge.
```

```
mysql> select * from studenttable;
+--------+-----------------+--------------------------+
| sid    | scity           | sname                    |
+--------+-----------------+--------------------------+
|     13 | Ballia          | Arjun                    |
|    123 | Ballia          | Shiva                    |
|    134 | Ballia          | Arjun                    |
|   1233 | Ballia          | Arjun                    |
|   1345 | Ballia          | Shiva                    |
|  13457 | Ballia          | Shiva                    |
| 134588 | New City Ballia | Shiva Updated with Merge |
+--------+-----------------+--------------------------+
7 rows in set (0.00 sec)
```

Delete the record in the table using delete and remove

delete() Method

- **Usage**: session.delete(entity)
- **Description**: This method was used to remove a persistent instance from the datastore. The argument could be an instance associated with the receiving Session or a transient instance with an identifier associated with an existing persistent state.
- **Deprecation:** The delete() method has been deprecated since Hibernate version 6.0. It is recommended to use the remove() method instead.
- **Cascading**: This operation cascades to associated instances if the association is mapped with cascade="delete".

remove() Method

- **Usage**: session.remove(entity)
- **Description**: This method is aligned with the JPA specification and is used to remove the given entity instance from the database.
- **Cascading**: Similar to delete(), it supports cascading if configured.
- **Recommendation:** Since Hibernate 6.0, remove() is the preferred method for deleting entities.

```
package com.telusko.app;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.telusko.model.Student;

public class LaunchDelete {
```

```
	public static void main(String[] args) {
		// TODO Auto-generated method stub

		SessionFactory sessionFactory = new Configuration().configure().

	addAnnotatedClass(Student.class).buildSessionFactory();
		Session session=null;
		Transaction transaction=null;
		boolean flag=false;

		try
		{
			session=sessionFactory.openSession();
			transaction=session.beginTransaction();
			Student st=new Student();
			st.setSid(3);
			st.setsName("Harsh");
			st.setScity("Jaipur");
//		session.persist(st);

			session.delete(st);
			// session.remove(st);

			flag=true;


		}
		catch(HibernateException e)
		{
			e.printStackTrace();
		}
		catch(Exception e)
		{
			e.printStackTrace();
```

```
            }
            finally
            {
                  if(flag==true)
                        transaction.commit();
                  else
                        transaction.rollback();

                  session.close();
                  sessionFactory.close();
            }

      }

}
```

Output:

```
mysql> select * from studenttable;
+-----+--------+-------+
| SID | SCITY  | SNAME |
+-----+--------+-------+
|   3 | Jaipur | Harsh |
+-----+--------+-------+
1 row in set (0.00 sec)

mysql> select * from studenttable;
Empty set (0.00 sec)
```

```
Zero Param Constructor for Hibernate
Hibernate:
    select
        null,
        s1_0.SNAME,
        s1_0.SCITY
    from
        StudentTable s1_0
    where
        s1_0.SID=?
Hibernate:
    delete
    from
        StudentTable
    where
        SID=?
```

Maven Project Update

By default when using eclipse as IDE, Hibernate or Maven projects may use JDK 1.5 if no specific JDK version is configured. However, you can update the JDK version by adding properties in the pom.xml file.

1. Default JDK Version (1.5):

- When Maven does not explicitly define the JDK version, it defaults to JDK 1.5.
- This can cause issues with newer features available in later versions of Java.

2. Updating the JDK Version:

- To update the JDK version for compiling your project, you need to specify the desired JDK version in the pom.xml file.
- You can do this by adding properties for maven.compiler.source and maven.compiler.target.

3. Properties to Update JDK Version:

To update the JDK version, add the following properties to your pom.xml:

```
<properties>
        <maven.compiler.source>14</maven.compiler.source>  <!-- Use JDK 14 for compiling -->
        <maven.compiler.target>14</maven.compiler.target>  <!-- Target JDK 14 -->
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> <!-- Ensures UTF-8 encoding -->
</properties>
```

4. Example pom.xml:

Here's an example pom.xml file with updated properties for JDK 14, along with Hibernate and MySQL dependencies:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
         <modelVersion>4.0.0</modelVersion>

         <groupId>com.example</groupId>
         <artifactId>HibernateFirst</artifactId>
         <version>0.1.0-SNAPSHOT</version>
         <name>HibernateFirst</name>

         <properties>
         <maven.compiler.source>14</maven.compiler.source>  <!-- Updated
JDK version -->
         <maven.compiler.target>14</maven.compiler.target>  <!-- Updated JDK
version -->
         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<!-- Encoding -->
         </properties>

         <dependencies>
         <!-- Hibernate Core Dependency -->
         <dependency>
         <groupId>org.hibernate.orm</groupId>
         <artifactId>hibernate-core</artifactId>
         <version>6.3.1.Final</version>
         </dependency>

         <!-- MySQL Connector Dependency -->
         <dependency>
         <groupId>mysql</groupId>
         <artifactId>mysql-connector-java</artifactId>
         <version>8.0.30</version>
         </dependency>
```

```
    </dependencies>
</project>
```

Summary:

- **Default JDK**: If not set, Maven projects use JDK 1.5.
- **Update JDK**: Use maven.compiler.source and maven.compiler.target properties to specify a newer JDK version (e.g., JDK 14).
- **Example**: The pom.xml above updates the project to use JDK 14, with dependencies for Hibernate and MySQL.

Selective Insertion with @Transient Annotation

The @Transient annotation in Java is used in JPA/Hibernate to mark a field of a class that should **not** be persisted into the database. Fields marked with @Transient are not included in any insert or update operations in the database. It is useful when you want certain fields to be ignored by JPA, either because they are calculated values or are used for business logic only.

Key Points:

1. **Ignore during Persistence**:
    - Fields marked with @Transient will **not be saved** to the database.
    - They will also **not be fetched** from the database.
2. **Non-persistent Field**:
    - These fields are used only in the Java object and **not part of the database schema**.
    - JPA ignores the annotated field for insert, update, or query operations.
3. **Common Use Cases**:
    - Fields used for business logic.
    - Calculated or derived fields (e.g., age based on dateOfBirth).
    - Temporary data, which doesn't need to be stored.
4. **Example**:

Student.java

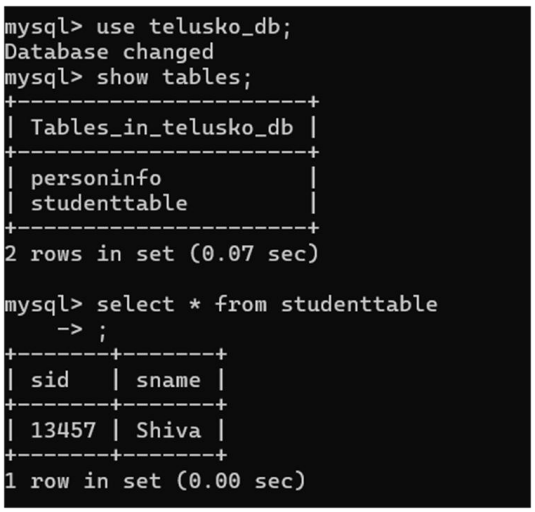
```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Transient;

@Entity
@Table(name="StudentTable")
public class Student {
	@Id
```

```
   @Column(name="sid")
		private int id;

   @Column(name="sname")
		private String name;

	@Transient
		private String city;


		public Student() {
			System.out.println("Zero param constructor");
		}

		public int getId() {
			return id;
		}
		public void setId(int id) {
			this.id = id;
		}
		public String getName() {
			return name;
		}
		public void setName(String name) {
			this.name = name;
		}
		public String getCity() {
			return city;
		}
		public void setCity(String city) {
			this.city = city;
		}

}
```

Main.java

Refer Insertion Notes

Output:



In the above example, the bonus field is annotated with @Transient and will not be saved in the database, even though it's part of the Java object.

5. **Difference from transient keyword**:
    - **@Transient**: Specifically for JPA (persistence framework) to ignore the field during database operations.
    - **transient keyword**: Used in Java for fields that should not be serialized (during Java object serialization).
6. **Important**:
    - @Transient fields can still be used in the application, but they will not be part of the database entity mapping.

Data Retrieval with get method

In Hibernate, the get() method is used to retrieve an entity from the database based on its primary key. It fetches the data and converts it into a Java object automatically. Here's a detailed explanation of how data retrieval works using Hibernate's get() method:

1. Entity Definition (Student Class)

First, we define the entity class that is mapped to the database table. In this case, the class Student is mapped to the StudentTable in the database.

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="StudentTable")
public class Student {
	@Id
	@Column(name="sid")
		private int id;

	@Column(name="sname")
		private String name;

	@Column(name="scity")
		private String city;

		public Student() {
				System.out.println("Zero param constructor");
		}
```

```
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
}
```

2. Data Flow During Retrieval

Here's how data retrieval happens step-by-step using get():

- **Step 1**: The get() method is called on the Session object. For example:

```
Student student = session.get(Student.class, 1);
```

- **Step 2**: Hibernate (ORM) takes care of the following:
    - It reads the **mapping details** from annotations (@Entity, @Table, @Column) to understand which table and columns to target.
    - It uses the configuration settings for the database connection.
- **Step 3**: Hibernate translates the get() method call into a SQL query using **JDBC**:

```
SELECT * FROM StudentTable WHERE sid=1;
```

- **Step 4**: **JDBC** executes the query on the database, retrieving the record.
- **Step 5**: Hibernate processes the result set. It finds the matching fields from the result set (sid, sname, scity) and maps them to the corresponding fields in the Student class.
- **Step 6**: Hibernate automatically creates an instance of the Student class and populates the fields with the data from the result set.
- **Step 7**: Hibernate calls the **zero-parameter constructor** (Student()) of the entity class and sets the values using setter methods (setId(), setName(), setCity()).

3. ResultSet to Object Mapping

Internally, Hibernate uses **ResultSet** from JDBC to get the data from the database. For each record in the ResultSet:

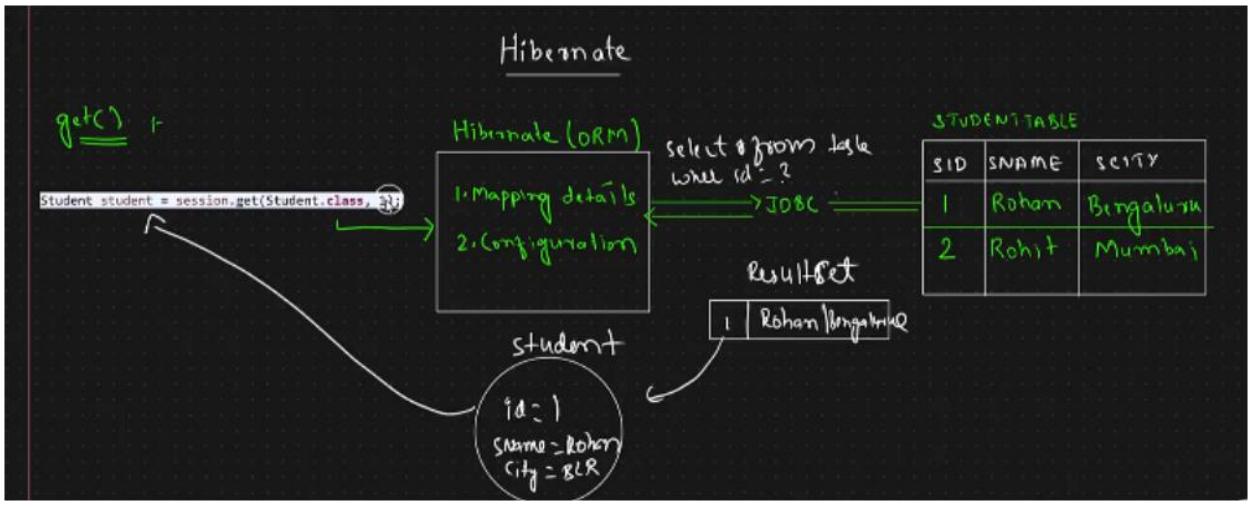
- It **calls the constructor** of the Student class to create an object.
- It **sets the values** of each field (id, name, city) using reflection and setter methods.

4. Main Class to Demonstrate Retrieval

Here's a simple main class that demonstrates how to retrieve data using Hibernate's get() method.

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class Main {
        public static void main(String[] args) {
        // Configure Hibernate
        SessionFactory factory = new
Configuration().configure("hibernate.cfg.xml")
                .addAnnotatedClass(Student.class).buildSessionFactory();

        // Open session
```

```
        Session session = factory.openSession();

        try {


        // Retrieve student with ID 1
        Student student = session.get(Student.class, 1);

        // Display the student details
          System.out.println("ID: " + student.getId());
          System.out.println("Name: " + student.getName());
          System.out.println("City: " + student.getCity());


        } finally {
        // Close session
          session.close();
          factory.close();
        }
        }
}
```



5. How ORM Works Behind the Scenes

- Hibernate simplifies the database interaction by mapping Java objects (entities) to database tables.
- The get() method abstracts away the complexity of SQL queries and JDBC code.
- **Automatic Object Creation**: Hibernate takes care of object creation from database records. You don't have to manually write code to parse the ResultSet.
- **Zero-Parameter Constructor**: Hibernate requires a no-arg constructor to create instances of the entity class.

6. Advantages of get() Method

- **Simple API**: The get() method is simple to use and returns null if no entity is found.
- **Eager Loading**: The get() method fetches the object immediately and returns a fully initialized entity.
- **Convenience**: Hibernate handles mapping and data retrieval without manual SQL queries.

Lazy Loading vs Eager Loading in Hibernate

In Hibernate, **lazy loading** and **eager loading** control how related entities are fetched from the database. The get(), load(), and getReference() methods play crucial roles in these processes, especially when dealing with proxy objects, transient objects, and the timing of data retrieval.

1. get() Method

- **Immediate Fetch** (Eager Loading): The get() method fetches the object from the database immediately, returning a fully initialized object.
- **Usage**: It is typically used when you need the actual object right away.
- **Return Type**: Returns null if no record is found.

Example:

```
Session session = sessionFactory.openSession();
Student student = session.get(Student.class, 1);  // Eagerly loads the object
System.out.println("ID: " + student.getId());
System.out.println("Name: " + student.getName());
System.out.println("City: " + student.getCity());
```

- **Behavior**: The get() method sends an immediate SQL SELECT query to the database to retrieve the record. The object is fully initialized when fetched.
- **Use Case**: Use get() when you need immediate access to the object, like when displaying student details.

2. load() Method

- **Lazy Loading**: The load() method doesn’t hit the database immediately. Instead, it returns a proxy object, and the actual database query is executed when you access a property of the entity.

- **Usage**: It is used when you might or might not need to access the object fields.
- **Return Type**: Throws an exception if the object is not found.

Example:

```
Session session = sessionFactory.openSession();
Student student = session.get(Student.class, 1);  // Eagerly loads the object
System.out.println("ID: " + student.getId());
System.out.println("Name: " + student.getName());
System.out.println("City: " + student.getCity());
```

- **Behavior**: load() returns a proxy object (a placeholder object). The actual query to the database is triggered only when a method like getName() or getCity() is called.
- **Proxy Object**: If you try to print the proxy object directly, it won't show the actual values until a method is accessed. When accessing the object's properties, Hibernate initializes the object by executing the query.
- **Use Case**: Use load() when you might not need the object immediately or in scenarios where performance optimization is needed by delaying the database call.

3. getReference() Method

- **Lazy Loading**: The getReference() method replaces the deprecated load() method in newer versions of Hibernate, maintaining the same lazy loading behavior by returning a proxy object.
- The actual query is executed only when the properties of the object are accessed.
- **Usage**: Typically used in entity relationships.
- **Return Type**: Throws an exception if the object is not found.

Example:

```
Session session = sessionFactory.openSession();
```

```
Student student = session.getReference(Student.class, 1);  // Proxy object
System.out.println("ID: " + student.getId());           // Executes query
```

- **Behavior**: Works similarly to load() by returning a proxy object. It triggers the database query only when the object properties are accessed.

4. Proxy Object Behavior

A **proxy object** is a placeholder object returned by Hibernate when using lazy loading (load() or getReference()). The proxy object doesn't contain the actual data; instead, the database is queried when an attribute (like getName() or getCity()) is accessed.

- **Accessing Proxy Object**: If the entity's properties (e.g., getName(), getCity()) are accessed, Hibernate triggers a database call to fetch the full object data. If the entity is **not found**, Hibernate throws an exception.
- **Proxy Object Example**:

```
Student student = session.load(Student.class, 1);  // Returns a proxy object
System.out.println("ID: " + student.getId());           // Triggers actual database call
```

5. Transient Object

A **transient object** in Hibernate is an object that has not been associated with any database session. It doesn't have a corresponding row in the database.

- **Behavior with Transient Objects**: If a transient object is used with methods like get() or load(), Hibernate will not find it in the session or database, which might lead to exceptions.

6. Key Differences Between get(), load(), and getReference()

- **get()**: Always hits the database immediately. Returns null if the object is not found.
- **load()**: Returns a proxy object. Throws an exception if the object is not found.

- **getReference()**: Similar to load(), returns a proxy object and throws an exception if the object is not found.

7. Example: Handling Proxy Object and Lazy Loading

```
Session session = sessionFactory.openSession();
try {
        // Using get()
        Student student = session.get(Student.class, 1);
        if (student != null) {
      System.out.println("ID: " + student.getId());
      System.out.println("Name: " + student.getName());
      System.out.println("City: " + student.getCity());
        } else {
      System.out.println("No data found with the given ID");
        }

        // Using load()
        //Student proxyStudent = session.load(Student.class, 2);  // Lazy loading
    Student proxyStudent = session.getReference(Student.class, 2);
        System.out.println("Accessing proxy object: " + proxyStudent.getName());

} catch (HibernateException e) {
   e.printStackTrace();
} finally {
        session.close();
}
```

Summary of Key Points:

- **get()**: Immediate data retrieval (eager loading), returns null if not found.
- **load() and getReference()**: Lazy loading, returns a proxy object, triggers data fetch only when properties are accessed, and throws exceptions if not found.

- **Proxy Object**: Used in lazy loading to delay the database query. Hibernate provides an actual instance only when properties are accessed.
- **Transient Objects**: Objects not yet associated with the session, potentially leading to exceptions if accessed in certain ways.
- **Lazy Initialization Exception**: Be careful when closing the session before accessing lazy-loaded properties.

Level 1 Caching in Hibernate

Level 1 (L1) Cache is the default cache mechanism provided by Hibernate for each session. It caches the objects during a session's lifespan, ensuring that multiple requests for the same object within a session do not result in multiple database queries. This cache is **session-scoped**, meaning it is only valid for the lifespan of a single session.

Key Points of Level 1 Cache:

1. **Session-Scope**: Each Hibernate session has its own Level 1 cache. Objects retrieved during a session are stored in that cache, and any subsequent requests for the same object will be served from the cache, avoiding unnecessary database hits.
2. **Caching Objects**: Objects are stored in the cache once they are retrieved using methods like get(), load(), or getReference(). If an object is already in the cache, Hibernate will return it directly without querying the database again.
3. **Automatic Cache Management**: Hibernate automatically manages the cache for the session. The user does not have to explicitly put or remove objects from the cache.
4. **Flush and Clear**: The cache gets cleared when the session is closed, or when the session is flushed or cleared manually. session.flush() writes changes to the database, and session.clear() removes all objects from the cache.

Flow of Data in Level 1 Cache:

- When a query is made for an object using get() or load(), Hibernate checks if the object is already present in the session's cache:
    - **If Present**: It returns the object from the cache (without hitting the database).
    - **If Absent**: It fetches the object from the database and stores it in the cache for future use.

Example: Level 1 Caching with Student EntityCode:

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Transient;

@Entity
@Table(name="StudentTable")
public class Student {
   @Id
   @Column(name="sid")
        private int id;

   @Column(name="sname")
        private String name;

   @Transient
        private String city;


        public Student() {
                System.out.println("Zero param constructor");
        }

        public int getId() {
                return id;
        }
        public void setId(int id) {
                this.id = id;
        }
        public String getName() {
                return name;
```

```
	}
	public void setName(String name) {
		this.name = name;
	}
	public String getCity() {
		return city;
	}
	public void setCity(String city) {
		this.city = city;
	}

	@Override
	public String toString() {
		return "Student [id=" + id + ", name=" + name + ", city=" + city +
"]";
	}

}
```

Main.java

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
```

```
public static void main(String[] args) {
    Configuration config = null;
    SessionFactory sessionFactory = null;
    Session session1 = null;
    Session session2=null;
    Transaction transaction = null;
    boolean flag = false;
    try {
        config = new Configuration();
        config.configure();
        sessionFactory = config.buildSessionFactory();
        session1 = sessionFactory.openSession();

        Student s1=session1.get(Student.class, 12);
        Student s2=session1.get(Student.class, 12);
        System.out.println(s1);
        System.out.println(s2);

        session2=sessionFactory.openSession();
        Student s3=session2.get(Student.class, 12);
        Student s4=session2.get(Student.class, 12);
        System.out.println(s3);
        System.out.println(s4);

    }  catch (HibernateException e) {
        System.out.println("Hibernate exception: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected exception: " + e.getMessage());
    } finally {

        if (session1 != null) {
            session1.close();
        }
        if (session2 != null) {
```

```
            session2.close();
        }
        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
  }
}
```

Output:

```
Hibernate:
    select
        s1_0.sid,
        s1_0.sname
    from
        StudentTable s1_0
    where
        s1_0.sid=?
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
Hibernate:
    select
        s1_0.sid,
        s1_0.sname
    from
        StudentTable s1_0
    where
        s1_0.sid=?
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
```

Notes:

1. For each session, only one call is made to the database for the same object, regardless of how many times the object is accessed.
2. When the session retrieves an object for the first time with a particular ID, the object is instantiated using the zero-argument constructor, and its values are populated using setter methods.

Explanation:

1. **Session 1**:
    - The first time session1.get(Student.class, 1) is called, Hibernate queries the database, retrieves the student object, and stores it in the **L1 cache** for session1.
    - When session1.get(Student.class, 1) is called again, Hibernate does **not hit the database**. It fetches the object from the cache and returns it.
2. **Session 2**:
    - A new session (session2) is opened, which means a new **L1 cache** is created for session2.
    - When session2.get(Student.class, 1) is called, Hibernate queries the database again because the cache is specific to session1. Objects in session1's cache are not available in session2.

Important Points to Remember:

1. **Each Session has its own Cache**: The cache is **session-specific**, so it does not persist across multiple sessions. Each session will have its own L1 cache.
2. **No Need for Configuration**: L1 cache is enabled by default, and no configuration is required.
3. **Cache Lifetime**: The L1 cache exists for the duration of a session. Once the session is closed, the cache is cleared.
4. **Flush and Clear**: You can clear the cache manually using session.clear(), or flush the session using session.flush().

Object Storage in Level 1 Cache:

- The **L1 cache** stores objects as **entity objects** mapped with their unique primary keys.
- The session checks the cache before querying the database, reducing unnecessary database interactions and improving performance.

Level 1 Cache Flow:

1. **Query Object**: The session.get() or session.load() is called.
2. **Check Cache**: Hibernate checks if the requested object is already present in the L1 cache of the session.
3. **Cache Hit**: If found, the object is returned from the cache.
4. **Cache Miss**: If not found, the database query is executed, the object is fetched and stored in the cache.

Output from Example:

1. **First session (session1)**:
    - First get() call fetches the object from the database and stores it in the session's cache.
    - Second get() call retrieves the object from the cache (no database hit).
2. **Second session (session2)**:
    - Since it's a new session, the cache is empty. A new database query is made to fetch the object.

- Notice how the database query is executed only once in **session1**. The subsequent call fetches the object from the cache. However, in **session2**, a new database query is made, showing that the L1 cache is session-specific.

Level 2 Cache in Hibernate(EhCache)

What is Level 2 Cache?

- Level 2 (L2) cache is a **shared cache** across sessions.
- Unlike the **first-level cache** (which is session-specific), L2 cache is used by all sessions, improving performance for **multiple requests**.
- It helps reduce **database hits** by storing objects in memory after the first fetch.

Enabling Level 2 Cache in Hibernate 5:

1. **Add Dependencies in pom.xml**:

For Hibernate 5:

```
<dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>5.6.15.Final</version>
</dependency>

<dependency>
        <groupId>org.ehcache</groupId>
        <artifactId>ehcache</artifactId>
        <version>3.10.8</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-ehcache -->
<dependency>
   <groupId>org.hibernate</groupId>
   <artifactId>hibernate-ehcache</artifactId>
   <version>5.6.15.Final</version>
</dependency>
```

Note:

- As of time when preparing this document, Hibernate 6 is still under development for full support of second-level caching with Ehcache.
- If you encounter errors related to the Ehcache version in your pom.xml, consider downgrading Ehcache to version 3.8.1.

2. **Hibernate Configuration:**
    - Enable L2 cache by adding properties in the configuration file (hibernate.cfg.xml):

```
<property
name="hibernate.cache.region.factory_class">org.hibernate.cache.ehcache.EhCa
cheRegionFactory</property>
<property name="hibernate.cache.use_second_level_cache">true</property>
```

3. **Annotated Entity Class**:
    - Use annotations in your **entity class** to specify caching strategies:
    - **@Cacheable** *// Marks the entity as cacheable*
      @Cache(usage = CacheConcurrencyStrategy.READ_ONLY) *// Specifies the cache strategy*

```
import javax.persistence.Cacheable;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
import javax.persistence.Transient;
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;
@Entity
@Table(name="StudentTable")
@Cacheable  // Marks the entity as cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_ONLY)  // Specifies the
```

cache strategy

```
public class Student {
  @Id
  @Column(name="sid")
        private int id;

  @Column(name="sname")
        private String name;

  @Transient
        private String city;


        public Student() {
                System.out.println("Zero param constructor");
        }

        public int getId() {
                return id;
        }
        public void setId(int id) {
                this.id = id;
        }
        public String getName() {
                return name;
        }
        public void setName(String name) {
                this.name = name;
        }
        public String getCity() {
                return city;
        }
        public void setCity(String city) {
                this.city = city;
```

```
	}
	@Override
	public String toString() {
		return "Student [id=" + id + ", name=" + name + ", city=" + city +
"]";
	}

}
```

- @Cacheable: Marks the entity as cacheable, allowing Hibernate to store it in the second-level cache.
- @Cache(usage = CacheConcurrencyStrategy.READ_WRITE): Defines how the cache will behave:
  i. READ_ONLY: Used for entities that never change.
  ii. READ_WRITE: Used for entities that may be modified by the application.

Main.java

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
  public static void main(String[] args) {
    Configuration config = null;
    SessionFactory sessionFactory = null;
    Session session1 = null;
    Session session2=null;
```

```
Transaction transaction = null;
boolean flag = false;
try {
   config = new Configuration();
   config.configure();
   sessionFactory = config.buildSessionFactory();
   session1 = sessionFactory.openSession();

   Student s1=session1.get(Student.class, 12);
   Student s2=session1.get(Student.class, 12);
   System.out.println(s1);
   System.out.println(s2);

   session2=sessionFactory.openSession();
   Student s3=session2.get(Student.class, 12);
   Student s4=session2.get(Student.class, 12);
   System.out.println(s3);
   System.out.println(s4);

}  catch (HibernateException e) {
   System.out.println("Hibernate exception: " + e.getMessage());
} catch (Exception e) {
   System.out.println("Unexpected exception: " + e.getMessage());
} finally {

   if (session1 != null) {
      session1.close();
   }
   if (session2 != null) {
      session2.close();
   }
   if (sessionFactory != null) {
      sessionFactory.close();
   }
```

```
    }
  }
}
```

Output:

```
Hibernate:
    select
        student0_.sid as sid1_0_0_,
        student0_.sname as sname2_0_0_
    from
        StudentTable student0_
    where
        student0_.sid=?
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
```

Key Points to Remember:

- **Ehcache** is one of the most popular **second-level cache providers** for Hibernate.
- You need to ensure the version compatibility between Hibernate and Ehcache.
- **Hibernate 5** supports Ehcache easily, but for **Hibernate 6**, the support is still in progress.

Hibernate Configuration Using Java Without XML file

In the provided code, we configure Hibernate directly in Java instead of using the traditional hibernate.cfg.xml file. Here's a breakdown of the important points with explanations.

Code:

```
Configuration config = null;
SessionFactory sessionFactory = null;
Session session = null;
Transaction transaction = null;
boolean flag = false;

config = new Configuration();

// Setting Hibernate properties directly using setProperty method
config.setProperty("hibernate.connection.driver_class",
"com.mysql.cj.jdbc.Driver");
config.setProperty("hibernate.connection.url",
"jdbc:mysql://localhost:3306/telusko_db");
config.setProperty("hibernate.connection.password", "root123");
config.setProperty("hibernate.connection.username", "root");
config.setProperty("hibernate.dialect", "org.hibernate.dialect.MySQLDialect");
config.setProperty("hibernate.hbm2ddl.auto", "update");
config.setProperty("hibernate.show_sql", "true");
config.setProperty("hibernate.format_sql", "true");

// Adding annotated class
config.addAnnotatedClass(Student.class);

// Building session factory
sessionFactory = config.buildSessionFactory();
```

```
// Opening a session
session = sessionFactory.openSession();
```

Explanation of Important Points:

1. **Configuration config = null;**:
    - **Purpose**: A Configuration object is used to specify all the configuration details that would normally be in the hibernate.cfg.xml file.
    - **Importance**: Here, we are not using XML files, so the configuration is directly defined in the Java code.

Advantages of Configuring Hibernate Without XML:

1. **Code-Centric Approach**: You avoid XML-based configuration, keeping everything within your code. This can be more intuitive for developers who prefer code over external configuration.
2. **Flexibility**: You can dynamically set properties or adjust them programmatically based on the environment (e.g., different database URLs for development and production).
3. **Simplified Management**: For smaller applications, it might simplify the setup since all the configuration is in one place, reducing the need for external files.

Key Notes:

- **Avoid Hardcoding**: It's good practice to avoid hardcoding sensitive information like database passwords. Consider using environment variables or a configuration management system.

Hibernate Configuration using hibernate.properties File

Steps to Configure Hibernate with MySQL:

1. **Create hibernate.properties file**:
    - The hibernate.properties file contains configuration settings to establish the connection with the MySQL database. The file should be placed in the src/main/resources folder of your project.

The properties in this file:

```
hibernate.connection.driver_class = com.mysql.cj.jdbc.Driver
hibernate.connection.url = jdbc:mysql://localhost:3306/telusko_db
hibernate.connection.username = root
hibernate.connection.password = root123
hibernate.dialect = org.hibernate.dialect.MySQLDialect
hibernate.hbm2ddl.auto = update
hibernate.show_sql = true
hibernate.format_sql = true
```

2. **Code to Automatically Identify hibernate.properties**:
    - In the Java code, you don't need to manually configure each setting using config.setProperty(). Hibernate can automatically detect the hibernate.properties file and use it to configure the database connection.

```
public static void main(String[] args) {
        Configuration config = new Configuration();
        SessionFactory sessionFactory = null;
        Session session = null;
        Transaction transaction = null;
```

```
        try {
        // Load hibernate.properties automatically
      config.configure();
      config.addAnnotatedClass(Student.class);

        sessionFactory = config.buildSessionFactory();
        session = sessionFactory.openSession();

        transaction = session.beginTransaction();

        // Perform your database operations here

      transaction.commit();
        } catch (Exception e) {
        if (transaction != null) {
        transaction.rollback();
        }
      e.printStackTrace();
        } finally {
        if (session != null) {
         session.close();
        }
        }
}
```

- config.configure(); loads the Hibernate settings from the hibernate.properties file.
- config.addAnnotatedClass(Student.class); tells Hibernate to look for entity annotations in the Student class.

2.**Key Concepts**:

- **hibernate.connection.driver_class**: Specifies the JDBC driver class.
- **hibernate.connection.url**: The database URL to connect to.

- **hibernate.connection.username/password**: Credentials for the database connection.
- **hibernate.dialect**: Hibernate uses this to generate SQL optimized for the specific database.
- **hibernate.hbm2ddl.auto**: Manages the schema update (update, create, create-drop, validate).
- **hibernate.show_sql**: If set to true, Hibernate will log the executed SQL queries.
- **hibernate.format_sql**: Pretty-print the SQL queries.

Exploring @GeneratedValue and @SequenceGenerator

1. @GeneratedValue Annotation:

- The @GeneratedValue annotation is used to define the strategy for generating primary key values automatically.
- There are different strategies you can use:
    - **GenerationType.IDENTITY**: This strategy lets the database handle the generation of primary keys, commonly used with auto-increment fields.
    - **GenerationType.SEQUENCE**: This strategy is used when a sequence is defined in the database, and Hibernate will use it to generate unique primary key values.
    - **GenerationType.TABLE**: A table is used to maintain the primary key generation.
    - **GenerationType.AUTO**: This allows Hibernate to automatically choose the strategy based on the database used.

Descriptions:

- @GeneratedValue(strategy=GenerationType.IDENTITY) indicates that Hibernate should rely on the auto-increment feature of the database for the primary key.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer id;
```

2. @SequenceGenerator Annotation:

- The @SequenceGenerator annotation is used in combination with GenerationType.SEQUENCE to specify a sequence in the database that Hibernate should use to generate primary key values.
- It defines:
    - **name**: The name of the sequence generator.
    - **sequenceName**: The name of the database sequence to use.
    - **initialValue**: The starting value of the sequence.
    - **allocationSize**: Defines the increment size of the sequence. It indicates how many values should be fetched at a time from the database sequence.

Descriptions:

- The custom sequence generator is defined with name="my_seq", which refers to a sequence called "My_OwnSequence", with an initial value of 100 and an allocation size of 1.

```
@GeneratedValue(generator = "my_seq", strategy =
GenerationType.SEQUENCE)
@SequenceGenerator(name = "my_seq", sequenceName =
"My_OwnSequence", initialValue = 100, allocationSize = 1)
private Integer id;
```

Example:

Student.java

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;
@Entity
@Table(name = "Studenttable")
public class Student {
   @Id
   @GeneratedValue(generator = "my_seq", strategy =
GenerationType.SEQUENCE)
   @SequenceGenerator(name = "my_seq", sequenceName =
"My_OwnSequence", initialValue = 100, allocationSize = 1)
   private Integer id;
   private String name;
   private String city;
   // Zero-argument constructor
   public Student() {
      System.out.println("Zero Param Constructor for Hibernate");
   }
   // Getters and setters
   public Integer getId() {
      return id;
   }
   public void setId(Integer id) {
      this.id = id;
   }
   public String getName() {
      return name;
   }
   public void setName(String name) {
      this.name = name;
   }
   public String getCity() {
```

```
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Configuration config = null;
        SessionFactory sessionFactory = null;
        Session session = null;
        Transaction transaction = null;
        boolean flag = false;
        try {
            config = new Configuration();
            config.configure();
            sessionFactory = config.buildSessionFactory();
            session = sessionFactory.openSession();
            Student student = new Student();

            student.setName("Arjun");
            student.setCity("Ballia");
            transaction = session.beginTransaction();
            session.persist(student);
            transaction.commit();
            System.out.println("Transaction commited Succesfully");
            flag = true;
        } catch (EntityExistsException e) {
            System.out.println("Entity already exists: " + e.getMessage());
        } catch (ConstraintViolationException e) {
```

```
            System.out.println("Constraint violation: " + e.getMessage());
        } catch (HibernateException e) {
            System.out.println("Hibernate exception: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected exception: " + e.getMessage());
        } finally {
           if(!flag)
           {
               if (transaction != null) {
                   transaction.rollback();
                   System.out.println("Transaction rolled back successfully");
               }
            }
            if (session != null) {
               session.close();
            }
            if (sessionFactory != null) {
               sessionFactory.close();
            }
        }
    }
}
```

Note: Never add primary key value if you autoincrement this result in Exception.

Output:

```
Hibernate:
    insert into My_OwnSequence values ( 100 )
Hibernate:
    create table Studenttable (
        id integer not null,
        city varchar(255),
        name varchar(255),
        primary key (id)
    ) engine=InnoDB
Zero Param Constructor for Hibernate
Hibernate:
    select
        next_val as id_val
    from
        My_OwnSequence for update
Hibernate:
    update
        My_OwnSequence
    set
        next_val= ?
    where
        next_val=?
Hibernate:
    insert
    into
        Studenttable
        (city, name, id)
    values
        (?, ?, ?)
Transaction commited Succesfully
```

```
mysql> select * from studenttable;
+-----+--------+-------+
| id  | city   | name  |
+-----+--------+-------+
| 100 | Ballia | Arjun |
+-----+--------+-------+
1 row in set (0.00 sec)
```

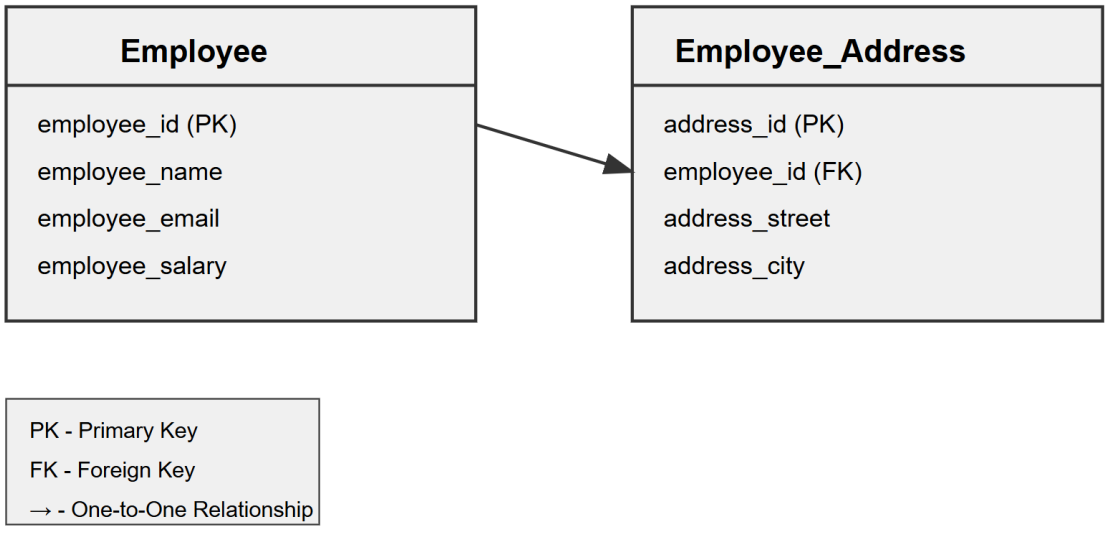
Summary:

- @GeneratedValue(strategy=GenerationType.IDENTITY): Auto-increments the primary key based on the database's ability to handle the ID generation.
- @SequenceGenerator: Used with GenerationType.SEQUENCE to specify a custom sequence for generating primary keys. This is helpful when you need more control over the primary key values, especially for databases that use sequences.

Introduction to Hibernate Association Mapping

- **Concept**: It's a way to tell Hibernate how your database tables are connected to each other.
- **Types of Relationships**:
    - **Primary Key**: Unique identifier for rows in the entity table.No two rows can have the same number.
    - **Foreign Key:** A field in one table that references or connects to the primary key of another table. It can be any data type (number, text, UUID, etc.) but must match the data type of the primary key it references.

Example: Employee and Employee_Address Tables Relationship

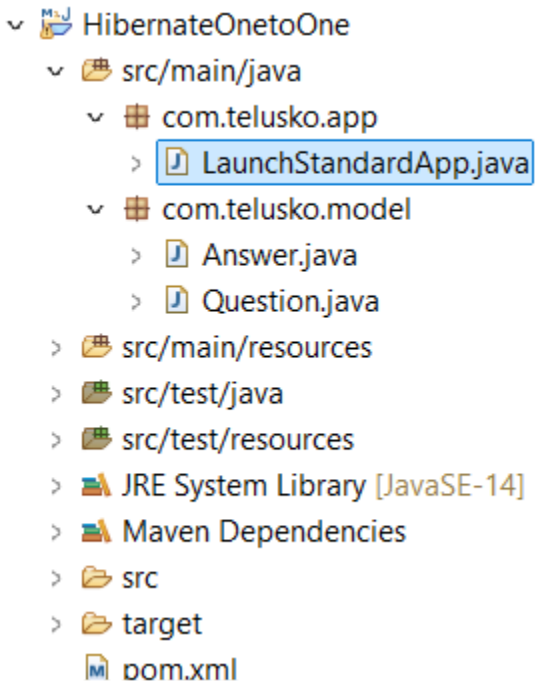


- In the Employee table, the column employee_id is a primary key.

- The Employee_Address table has address_id as the primary key, while employee_id is a foreign key referencing the Employee table.

Hibernate One-to-One Association

- **Definition**: A one-to-one association refers to a relationship where one entity instance is associated with exactly one instance of another entity.
- Project Structure



Example in Code:

Student.java

```
package com.telusko.model;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
```

```
public class Question
{
	@Id
	@Column(name="question_id")
    private Integer id;

    private String question;

    @OneToOne(cascade=CascadeType.ALL)
    private Answer answer;

    public Question()
    {
	 System.out.println("Zero Param Constructor of Question");
    }

	@Override
	public String toString() {
		return "Question [id=" + id + ", question=" + question + ", answer="
+ answer + "]";
	}

	public Integer getId() {
		return id;
	}

	public void setId(Integer id) {
		this.id = id;
	}

	public String getQuestion() {
		return question;
	}
```

```
	public void setQuestion(String question) {
		this.question = question;
	}

	public Answer getAnswer() {
		return answer;
	}

	public void setAnswer(Answer answer) {
		this.answer = answer;
	}


}
```

Answer.java

```
package com.telusko.model;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
public class Answer
{
	@Id
	@Column(name="answer_id")
    private Integer id;
    private String answer;
```

```
    public Question getQuestion() {
            return question;
        }
        public void setQuestion(Question question) {
            this.question = question;
        }

        @OneToOne(cascade=CascadeType.ALL)
    private Question question;

    public Answer()
    {
         System.out.println("Zero param Constructor of Answer");
    }
        public Integer getId() {
            return id;
        }

        public void setId(Integer id) {
            this.id = id;
        }

        public String getAnswer() {
            return answer;
        }

        public void setAnswer(String answer) {
            this.answer = answer;
        }
        @Override
        public String toString() {
            return "Answer [id=" + id + ", answer=" + answer + ", question=" +
question + "]";
```

```
	}


}
```

```
package com.telusko.app;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.telusko.model.Answer;
import com.telusko.model.Question;

public class LaunchStandardApp
{

	public static void main(String[] args)
	{
		Configuration config = null;
		SessionFactory sessionFactory = null;
		Session session = null;
		Transaction transaction = null;
		boolean flag=false;

		config=new Configuration();

		config.configure();

		sessionFactory=config.buildSessionFactory();
```

```
            session=sessionFactory.openSession();

            Question q1=new Question();
            q1.setId(1);
            q1.setQuestion("What is Hibernate?");

            Answer answer1 =new Answer();
            answer1.setId(1);
            answer1.setAnswer("Hibernate is an ORM Framewok");

            q1.setAnswer(answer1);
            answer1.setQuestion(q1);



            Question q2=new Question();
            q2.setId(2);
            q2.setQuestion("What is JPA?");

            Answer answer2 =new Answer();
            answer2.setId(2);
            answer2.setAnswer("It is a specificatiion used to persist "
                        + "data between Java object and relational database.");

            q2.setAnswer(answer2);
            answer2.setQuestion(q2);

//          Question qa=session.get(Question.class, 1);
//          System.out.println(qa);

            try
            {
                transaction=session.beginTransaction();
                //session.save(student);
```

```
                    session.persist(q1);
                    session.persist(q2);

//                  session.persist(answer1);
//                  session.persist(answer2);
                  flag=true;

                }
                catch(HibernateException e)
                {
                    e.printStackTrace();
                }
                catch(Exception e)
                {
                    e.printStackTrace();
                }

                finally
                {
                    if(flag==true)
                    {
                        transaction.commit();
                    }
                    else
                    {
                        transaction.rollback();
                    }

                    session.close();
                    sessionFactory.close();

                }

        }
```

```
}
```

Output:

```
mysql> drop database telusko_db;
Query OK, 6 rows affected (0.10 sec)

mysql> create database telusko_db;
Query OK, 1 row affected (0.01 sec)

mysql> show tables;
ERROR 1046 (3D000): No database selected
mysql> use telusko_db;
Database changed
mysql> show tables;
+----------------------+
| Tables_in_telusko_db |
+----------------------+
| answer               |
| question             |
+----------------------+
2 rows in set (0.00 sec)
```

```
mysql> select * from question;
+------------------+-------------+--------------------+
| answer_answer_id | question_id | question           |
+------------------+-------------+--------------------+
|                1 |           1 | What is Hibernate? |
|                2 |           2 | What is JPA?       |
+------------------+-------------+--------------------+
2 rows in set (0.00 sec)

mysql> select * from answer;
+-----------+---------------------+-----------------------------------------------------------------------------------------------------+
| answer_id | question_question_id | answer
      |
+-----------+---------------------+-----------------------------------------------------------------------------------------------------+
|         1 |                   1 | Hibernate is an ORM Framewok
     |
|         2 |                   2 | It is a specificatiion used to persist data between Java object and relational database. |
+-----------+---------------------+-----------------------------------------------------------------------------------------------------+
2 rows in set (0.00 sec)
```

```
Hibernate:
    drop table if exists Answer
Hibernate:
    drop table if exists Question
Hibernate:
    create table Answer (
        answer_id integer not null,
        question_question_id integer,
        answer varchar(255),
        primary key (answer_id)
    ) engine=InnoDB
Sept 15, 2024 12:23:41 PM org.hibernate.resource.transaction.backend.jdbc.internal.D
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engi
Hibernate:
    create table Question (
        answer_answer_id integer,
        question_id integer not null,
        question varchar(255),
        primary key (question_id)
    ) engine=InnoDB
Hibernate:
    alter table Answer
       add constraint UK_ljqm891ywk3128u2dyfkmc69d unique (question_question_id)
Hibernate:
    alter table Question
       add constraint UK_9y1pwogen3tjh3k2r2spard44 unique (answer_answer_id)
```

```
Hibernate:
    alter table Answer
       add constraint FK8yiifgngf37mxn437tqdab3rg
       foreign key (question_question_id)
       references Question (question_id)
Hibernate:
    alter table Question
       add constraint FKs6ghcwuovtcp489oo5dy7rvk5
       foreign key (answer_answer_id)
       references Answer (answer_id)
Zero Param Constructor of Question
Zero param Constructor of Answer
Zero Param Constructor of Question
Zero param Constructor of Answer
```

```
Hibernate:
    insert
    into
        Answer
        (answer, question_question_id, answer_id)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        Question
        (answer_answer_id, question, question_id)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        Answer
        (answer, question_question_id, answer_id)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        Question
        (answer_answer_id, question, question_id)
    values
        (?, ?, ?)
```

```
Hibernate:
    update
        Answer
    set
        answer=?,
        question_question_id=?
    where
        answer_id=?
Hibernate:
    update
        Answer
    set
        answer=?,
        question_question_id=?
    where
        answer_id=?
```

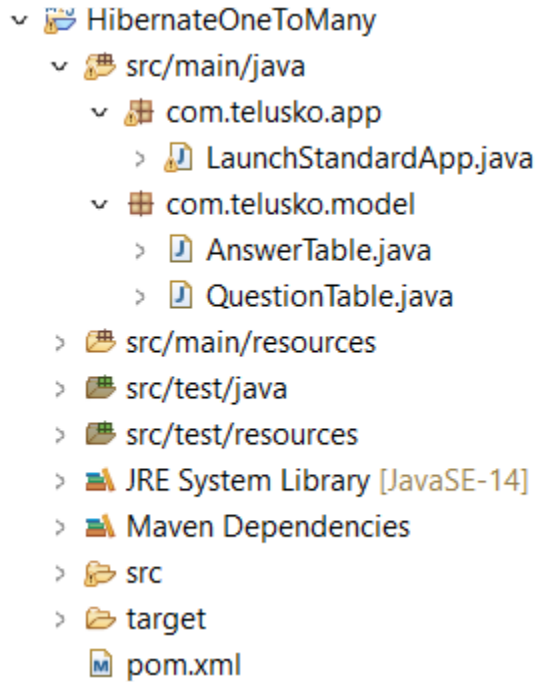
Explanation:

- The @OneToOne annotation specifies a one-to-one relationship between Question and Answer.

- CascadeType.ALL: Operations like save or delete performed on the Question entity are cascaded to the Answer entity.
- If CascadeType.ALL is not used, you need to save the Answer object separately.
- This relationship can work as either unidirectional or bidirectional.

Hibernate One-to-Many & Many-to-One Association

- **Definition**: A one-to-many association refers to a relationship where one instance of an entity can be associated with multiple instances of another entity.
- **Project Structure**



Example in Code:

```
package com.telusko.model;

import java.util.List;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
```

```
@Entity
public class QuestionTable
{
        @Id
        @Column(name="question_id")
    private Integer id;

    private String question;

   @OneToMany(cascade=CascadeType.ALL)
    private List<AnswerTable> answerList;

    public QuestionTable()
    {
        System.out.println("Zero Param Constructor of Question");
    }



        @Override
        public String toString() {
                return "QuestionTable [id=" + id + ", question=" + question + ",
answerList=" + answerList + "]";
        }



        public Integer getId() {
                return id;
        }

        public void setId(Integer id) {
                this.id = id;
        }
```

```
	public String getQuestion() {
		return question;
	}

	public void setQuestion(String question) {
		this.question = question;
	}

	public List<AnswerTable> getAnswerList() {
		return answerList;
	}

	public void setAnswerList(List<AnswerTable> answerList) {
		this.answerList = answerList;
	}
}
```

```
package com.telusko.model;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;

@Entity
public class AnswerTable
{
	@Id
	@Column(name="answer_id")
    private Integer id;
```

```
private String answer;

@ManyToOne(cascade=CascadeType.ALL)
private QuestionTable questionTable;

public QuestionTable getQuestion() {
		return questionTable;
	}


public void setQuestionTable(QuestionTable questionTable) {
		this.questionTable = questionTable;
	}


	public AnswerTable()
{
	 System.out.println("Zero param Constructor of Answer");
}
	public Integer getId() {
		return id;
	}

	public void setId(Integer id) {
		this.id = id;
	}

	public String getAnswer() {
		return answer;
	}

	public void setAnswer(String answer) {
		this.answer = answer;
	}
```

```
	@Override
	public String toString() {
		return "AnswerTable [id=" + id + ", answer=" + answer + ",
questionTable=" + questionTable + "]";
	}

}
```

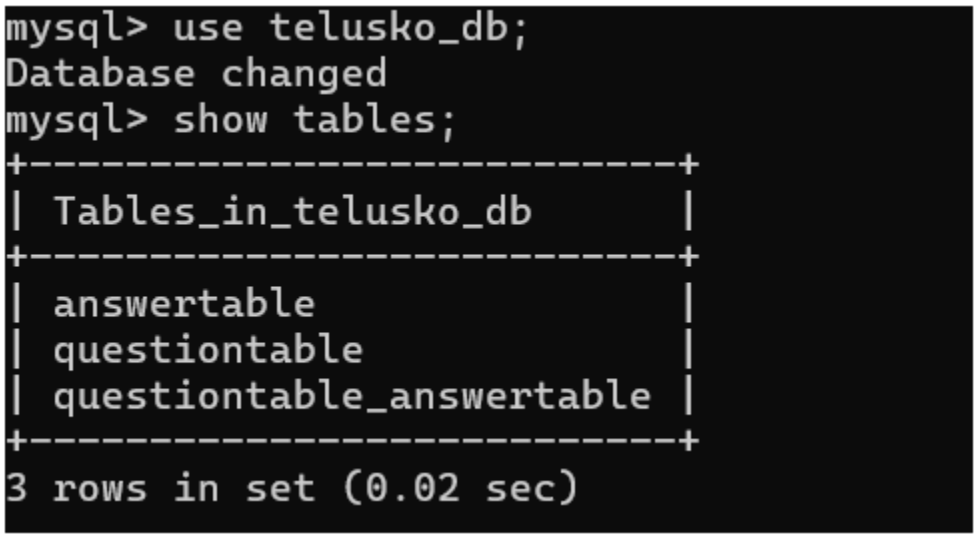
```
package com.telusko.app;

import java.util.ArrayList;
import java.util.List;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.telusko.model.AnswerTable;
import com.telusko.model.QuestionTable;

public class LaunchStandardApp
{

	public static void main(String[] args)
	{
		Configuration config = null;
		SessionFactory sessionFactory = null;
		Session session = null;
		Transaction transaction = null;
```

```
            boolean flag=false;

            config=new Configuration();

            config.configure();

            sessionFactory=config.buildSessionFactory();

            session=sessionFactory.openSession();

            QuestionTable q1=new QuestionTable();
            q1.setId(1);
            q1.setQuestion("What is Hibernate?");

            AnswerTable answer1 =new AnswerTable();
            answer1.setId(1);
            answer1.setAnswer("Hibernate is an ORM Framewok");
            answer1.setQuestionTable(q1);

            AnswerTable answer2 =new AnswerTable();
            answer2.setId(2);
            answer2.setAnswer("Hibernate is an implementation of JPA");
            answer2.setQuestionTable(q1);

            List<AnswerTable> answers=new ArrayList<AnswerTable>();
            answers.add(answer1);
            answers.add(answer2);

            q1.setAnswerList(answers);

//          QuestionTable question = session.get(QuestionTable.class,1);
//
//          System.out.println(question.getQuestion());
////        answer=question.getAnswerList();
```

```
//
//              for(AnswerTable answers: question.getAnswerList())
//              {
//                      System.out.println(answers.getAnswer());
//              }

                try
                {
        transaction=session.beginTransaction();

                        session.persist(q1);


                  flag=true;

                }
                catch(HibernateException e)
                {
                        e.printStackTrace();
                }
                catch(Exception e)
                {
                        e.printStackTrace();
                }

                finally
                {
        if(flag==true)
                {
                        transaction.commit();
                }
                else
                {
                        transaction.rollback();
```

```
                }

                session.close();
                sessionFactory.close();

            }

        }

}
```

Output:



```
mysql> select * from answertable;
+-----------+--------------------------+--------------------------------------+
| answer_id | questionTable_question_id | answer                               |
+-----------+--------------------------+--------------------------------------+
|         1 |                        1 | Hibernate is an ORM Framewok         |
|         2 |                        1 | Hibernate is an implementation of JPA |
+-----------+--------------------------+--------------------------------------+
2 rows in set (0.00 sec)

mysql> select * from questiontable;
+-------------+--------------------+
| question_id | question           |
+-------------+--------------------+
|           1 | What is Hibernate? |
+-------------+--------------------+
1 row in set (0.00 sec)
```

```
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.en
Hibernate:
    create table QuestionTable (
        question_id integer not null,
        question varchar(255),
        primary key (question_id)
    ) engine=InnoDB
Hibernate:
    create table QuestionTable_AnswerTable (
        QuestionTable_question_id integer not null,
        answerList_answer_id integer not null
    ) engine=InnoDB
Hibernate:
    alter table QuestionTable_AnswerTable
       add constraint UK_npi4sdg5u0tld2fpyi2ky0f1n unique (answerList_answer_id)
Hibernate:
    alter table AnswerTable
       add constraint FKnjhb8cv0jqswrceb2rwmtn2ik
       foreign key (questionTable_question_id)
       references QuestionTable (question_id)
Hibernate:
    alter table QuestionTable_AnswerTable
       add constraint FK6511gddj5t07otv3ld2wj8ki5
       foreign key (answerList_answer_id)
       references AnswerTable (answer_id)
Hibernate:
    alter table QuestionTable_AnswerTable
       add constraint FK1jck3814ug4p33ao7wx2iob5q
       foreign key (QuestionTable_question_id)
       references QuestionTable (question_id)
Zero Param Constructor of Question
Zero param Constructor of Answer
Zero param Constructor of Answer
```

```
Hibernate:
    insert
    into
        QuestionTable
        (question, question_id)
    values
        (?, ?)
Hibernate:
    insert
    into
        AnswerTable
        (answer, questionTable_question_id, answer_id)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        AnswerTable
        (answer, questionTable_question_id, answer_id)
    values
        (?, ?, ?)
Hibernate:
    insert
    into
        QuestionTable_AnswerTable
        (QuestionTable_question_id, answerList_answer_id)
    values
        (?, ?)
Hibernate:
    insert
    into
        QuestionTable_AnswerTable
        (QuestionTable_question_id, answerList_answer_id)
    values
        (?, ?)
```

- **Explanation**:
    - The @OneToMany annotation defines a one-to-many relationship between QuestionTable and AnswerTable.
    - CascadeType ALL ensures that operations like saving or deleting a QuestionTable object are cascaded to the related AnswerTable objects.
    - You can use session.get(QuestionTable.class, id) to retrieve the QuestionTable object and associated answers.

- **Explanation**:
  - @ManyToOne is used to map the relationship from AnswerTable to QuestionTable.
  - Each AnswerTable instance corresponds to one QuestionTable entity.
  -

Hibernate Many-to-Many Association

- **Definition**: A many-to-many association refers to a relationship where multiple instances of one entity are associated with multiple instances of another entity.

Example in Code:

```
package com.telusko.model;

import java.util.Set;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToMany;

@Entity
public class Students
{
        @Id
        @Column(name="students_id")
    private Integer id;

        private String name;

        private String city;

        @ManyToMany(cascade=CascadeType.ALL)
        private Set<Courses> courses;

          public Set<Courses> getCourses() {
              return courses;
```

```
}

public void setCourses(Set<Courses> courses) {
        this.courses = courses;
}

        public Students()
    {
         System.out.println("Zero Param Constructor of Students ");
    }

        public Integer getId() {
                return id;
        }

        public void setId(Integer id) {
                this.id = id;
        }

        public String getName() {
                return name;
        }

        public void setName(String name) {
                this.name = name;
        }

        public String getCity() {
                return city;
        }

        public void setCity(String city) {
                this.city = city;
        }
```

```
	@Override
	public String toString() {
		return "Students [id=" + id + ", name=" + name + ", city=" +
city + ", courses=" + courses + "]";
	}


}
```

```
package com.telusko.model;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;

@Entity
public class Courses
{
	@Id
	@Column(name="courses_id")
    private Integer id;

    private String courseName;

    private Integer coursePrice;

    public Courses()
    {
	 System.out.println("Zero Param Constructor of Courses");
    }
```

```
	public Integer getCoursePrice() {
		return coursePrice;
	}

	public void setCoursePrice(Integer coursePrice) {
		this.coursePrice = coursePrice;
	}

	public Integer getId() {
		return id;
	}

	public void setId(Integer id) {
		this.id = id;
	}

	public String getCourseName() {
		return courseName;
	}

	public void setCourseName(String courseName) {
		this.courseName = courseName;
	}

	@Override
	public String toString() {
		return "Courses [id=" + id + ", courseName=" + courseName + ",
coursePrice=" + coursePrice + "]";
	}

}
```

```
package com.telusko.app;

import java.util.HashSet;
import java.util.Set;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.telusko.model.Courses;
import com.telusko.model.Students;

public class LaunchStandardApp
{

	public static void main(String[] args)
	{
		Configuration config = null;
		SessionFactory sessionFactory = null;
		Session session = null;
		Transaction transaction = null;
		boolean flag=false;

		config=new Configuration();

		config.configure();

		sessionFactory=config.buildSessionFactory();

		session=sessionFactory.openSession();

		Courses c1=new Courses();
```

```
c1.setId(1);
c1.setCourseName("Java");
c1.setCoursePrice(1999);

Courses c2=new Courses();
c2.setId(2);
c2.setCourseName("Hibernate");
c2.setCoursePrice(499);

Courses c3=new Courses();
c3.setId(3);
c3.setCourseName("SpringBoot");
c3.setCoursePrice(999);

Set<Courses> set1=new HashSet<>();
set1.add(c1);
set1.add(c2);
set1.add(c3);

Set<Courses> set2=new HashSet<>();
set2.add(c1);
set2.add(c2);


Students s1 = new Students();
s1.setId(1);
s1.setName("Ramesh");
s1.setCity("Chennai");
s1.setCourses(set1);

Students s2 = new Students();
s2.setId(2);
s2.setName("Harsh");
s2.setCity("Jaipur");
```

```
            s2.setCourses(set2);

            Students s3 = new Students();
            s3.setId(3);
            s3.setName("Shushil");
            s3.setCity("Mumbai");
            s3.setCourses(set1);

//
//            Students stu1 = session.get(Students.class, 1);
//            System.out.println(stu1);
//            System.out.println("*****************************");
//
//            Students stu2 = session.get(Students.class, 2);
//            System.out.println(stu2);
//
//            System.out.println("*****************************");
//
//            Students stu3 = session.get(Students.class, 3);
//            System.out.println(stu3);
//


            try
            {
                  transaction=session.beginTransaction();
                  session.persist(s1);
                  session.persist(s2);
                  session.persist(s3);
              flag=true;

            }
            catch(HibernateException e)
            {
                  e.printStackTrace();
```

```
            }
            catch(Exception e)
            {
                  e.printStackTrace();
            }

            finally
            {
                  if(flag==true)
                  {
                        transaction.commit();
                  }
                  else
                  {
                        transaction.rollback();
                  }

                  session.close();
                  sessionFactory.close();

            }

      }

}
```

Output:

```
mysql> create database telusko_db;
Query OK, 1 row affected (0.02 sec)

mysql> use telusko_db;
Database changed
mysql> show tables;
+---------------------+
| Tables_in_telusko_db |
+---------------------+
| courses             |
| students            |
| students_courses    |
+---------------------+
3 rows in set (0.01 sec)
```

```
mysql> select * from courses;
+------------+------------+-------------+
| courses_id | courseName | coursePrice |
+------------+------------+-------------+
|          1 | Java       |        1999 |
|          2 | Hibernate  |         499 |
|          3 | SpringBoot |         999 |
+------------+------------+-------------+
3 rows in set (0.00 sec)

mysql> select * from students;
+-------------+---------+---------+
| students_id | city    | name    |
+-------------+---------+---------+
|           1 | Chennai | Ramesh  |
|           2 | Jaipur  | Harsh   |
|           3 | Mumbai  | Shushil |
+-------------+---------+---------+
3 rows in set (0.00 sec)

mysql> select * from students_courses;
+----------------------+--------------------+
| Students_students_id | courses_courses_id |
+----------------------+--------------------+
|                    1 |                  1 |
|                    2 |                  1 |
|                    3 |                  1 |
|                    1 |                  2 |
|                    2 |                  2 |
|                    3 |                  2 |
|                    1 |                  3 |
|                    3 |                  3 |
+----------------------+--------------------+
```

- **Explanation**:
  - The @ManyToMany annotation maps a many-to-many relationship between Student and Course.
  - A Set<Course> is used to represent the list of courses a student is enrolled in.

    - CascadeType ALL ensures that all operations performed on the Student entity cascade to the associated Course entities.

Use Cases:

- Multiple students can enroll in multiple courses, and each course can have multiple students.

Working with LOBs

1. What is a LOB?

LOB (Large Object) is a data type used to store large amounts of data in databases. It can be divided into:

- **BLOB (Binary Large Object)**:
  - Stores binary data such as images, audio, and video files.
  - Represented as byte[] in Java.
- **CLOB (Character Large Object)**:
  - Stores large text data like documents, XML, or HTML.
  - Represented as char[] or String in Java.

2. How to Read Objects (BLOB and CLOB)

Reading a BLOB

```
FileInputStream fis = new
FileInputStream("D:\\IO\\Java.JPG");
image = new byte[fis.available()];
fis.read(image);
```

  - **FileInputStream**:
    - Used to read binary data from a file.
    - new FileInputStream("D:\\IO\\Java.JPG"): Opens a stream to read the file Java.JPG.
  - **fis.available()**:
    - Returns the number of bytes available in the file for reading.
    - Allocates a byte[] array of the required size.
  - **fis.read(image)**:
    - Reads the file's content into the byte[] array.

Reading a CLOB

```
File file = new File("D:\\IO\\PersonalInfo.txt");
reader = new FileReader(file);
textFile = new char[(int) file.length()];
reader.read(textFile);
```

- **File**:
  - Represents the text file PersonalInfo.txt.
  - new File("D:\\IO\\PersonalInfo.txt"): Points to the file location.
- **FileReader**:
  - Used to read character data from a file.
  - new FileReader(file): Opens the file for reading.
- **file.length()**:
  - Returns the file size in bytes, cast to int to create a char[].
- **reader.read(textFile)**:
  - Reads the file's content into the char[] array.

3. Writing Retrieved Data to Local System

Writing BLOB Data

```
FileOutputStream fos = new FileOutputStream("Java.JPG");
fos.write(studentInfo.getImage());
```

- **FileOutputStream**:
  - Used to write binary data to a file.
  - new FileOutputStream("Java.JPG"): Opens a stream to write to the file Java.JPG.
- **fos.write(studentInfo.getImage())**:
  - Writes the binary data (image) from the studentInfo object into the file.

Writing CLOB Data

```
FileWriter writer = new FileWriter("PersonalInfo.txt");
writer.write(studentInfo.getTextFile());
```

- **FileWriter**:
  - Used to write character data to a file.
  - new FileWriter("PersonalInfo.txt"): Opens a stream to write to the file PersonalInfo.txt.
- **writer.write(studentInfo.getTextFile())**:
  - Writes the character data (text) from the studentInfo object into the file.

4. Use of Annotations in the Code

@Lob

- Indicates that the field should be mapped as a large object in the database.
- Automatically maps:
  - **byte[]** to a BLOB column.
  - **char[]** **or** **String** to a CLOB column.

@Column(length=100000)

- Sets a column-specific attribute in the database.
- **length=100000**:
  - Specifies the maximum size of the column for storing the BLOB.

Entity Class Example:

```
@Lob
@Column(length = 100000)
private byte[] image;

@Lob
private char[] textFile;
```

- **image**:
  - Stores binary data as a BLOB.
  - Limited to a maximum of 100,000 bytes.
- **textFile**:
  - Stores character data as a CLOB.

5. Step-by-Step Summary

1. **Reading BLOB and CLOB**:
   - Use FileInputStream and FileReader to read the data from files.
2. **Writing Data Locally**:
   - Use FileOutputStream and FileWriter to save the data to files.
3. **Database Mapping**:
   - Use @Lob for large objects and configure with @Column for custom sizes.
4. **Stream Management**:
   - Always close streams after usage to avoid resource leaks.

Introduction to HQL (JPQL)

Introduction to HQL (Hibernate Query Language)

HQL (Hibernate Query Language) is a powerful query language used with Hibernate and JPA to interact with the database in a simplified and object-oriented way. It operates on **entities** and **their attributes** rather than directly on database tables and columns. This abstraction makes HQL database-independent and easier to use for Java developers.

Key Features of HQL

1. **Entity-Based Queries**
   - Unlike SQL, which works directly with tables and columns, HQL uses **class names** and **field names** defined in your Java entities.
   - This approach keeps the queries consistent with your Java application's object model.
2. **Database Independence**
   - HQL abstracts SQL queries, making your application more portable across different databases.
3. **Bulk Operations**
   - HQL supports executing operations like `INSERT`, `UPDATE`, and `DELETE` on multiple rows in one go.

Basic HQL and SQL Syntax Examples

1. Selecting Data

Context: Retrieve details of an employee named "Navin Reddy."

SQL Query:

```
SELECT * FROM EMP WHERE FNAME='Navin' AND
LNAME='Reddy';
```

HQL Query:

```
FROM Employee WHERE FNAME='Navin' AND LNAME='Reddy';
```

2. Deleting Data

Context: Delete an employee record with the unique ID of 7.

SQL Query:

```
DELETE FROM EMP WHERE ID=7;
```

HQL Query:

```
DELETE FROM Employee WHERE ID=7;
```

3. Updating Data

Context: Update the salary of an employee with the ID of 7 to 50,000.

SQL Query:

```
UPDATE EMP SET SALARY=50000 WHERE ID=7;
```

HQL Query:

```
UPDATE Employee SET SALARY=50000 WHERE ID=7;
```

Why Use HQL?

HQL simplifies database operations by keeping the queries consistent with your Java code. It helps you write cleaner and more maintainable queries while ensuring portability across different database systems.

Data Retrieval with HQL

Basic HQL Query to Retrieve Data:

When we want to get data from our database using Hibernate, we can use HQL.

```
Session session1 = null;
try {
        session1 = sessionFactory.openSession();
   Query<Student> query = session1.createQuery("FROM Student",
Student.class);
   List<Student> listStudent = query.list();
        for (Student s : listStudent) {
       System.out.println(s);
        }
} catch (HibernateException e) {
   System.out.println(e);
}
```

Explanation:

1. First, we need a Session
    - A Session is like a connection to your database
    - We create it using `sessionFactory.openSession()`
2. The Query Process:
    - The line `createQuery("FROM Student", Student.class)` tells Hibernate:
        - "Hey, get me all records from the Student table"
        - `Student.class` tells Hibernate to convert the results into Student objects
3. Getting Results:
    - `query.list()` executes the query and gives us back a List of Student objects
    - We can then loop through this list to see each Student's data
4. Error Handling:

- The try-catch block helps us handle any database errors gracefully
- If something goes wrong, the HibernateException will catch it

More on Data Retrieval with HQL

Parameterized Queries: Used to avoid SQL injection and pass variables to the query.

```
Query<Student> query = session1.createQuery("FROM Student WHERE city=:city", Student.class);
query.setParameter("city", "Bengaluru");
List<Student> listStudent = query.list();
```

Using Multiple Parameters in HQL:

```
Query query = session1.createQuery("SELECT sname FROM Student WHERE city IN(:city1, :city2)");
query.setParameter("city1", "Chennai");
query.setParameter("city2", "Bengaluru");
List<String> listStudent = query.list();
```

Explanation:

- Use setParameter() to pass values to the query.
- Retrieve specific fields like sname instead of entire objects.
- HQL can perform IN queries to fetch records based on multiple conditions.

Delete and Update Operations

Updating Records in HQL:

```
session = sessionFactory.openSession();
transaction = session.beginTransaction();
Query query = session.createQuery("UPDATE Student SET scity=:city WHERE sid=:id");
query.setParameter("city", "Chennai");
query.setParameter("id", 2);
query.executeUpdate();
transaction.commit();
session.close();
```

Deleting Records in HQL:

```
session = sessionFactory.openSession();
transaction = session.beginTransaction();
Query query = session.createQuery("DELETE FROM Student WHERE sid=:id");
query.setParameter("id", 2);
query.executeUpdate();
transaction.commit();
session.close();
```

Explanation:

- Open session and begin transaction.
- Use executeUpdate() for both UPDATE and DELETE operations.
- Use setParameter() to pass dynamic values for queries.
- Close the session and commit the transaction after completion.